Tcl/Tk on Linux

X11, XDG, locale, Xfce, tests under Xvfb

Gregor Ebbing

4 November 2026

Contents

1 What this book is **7**
1.1 Introduction 7
1.2 What the examples work with 7
1.3 How to read the book 8
2 tcl_platform and the distribution **8**
2.1 Introduction 8
2.2 The keys 8
2.2.1 threaded under Tcl 9 9
2.2.2 osVersion is the kernel 9
2.2.3 user is not USER 9
2.3 Distribution: /etc/os-release 9
2.4 The running process: /proc/self 10
2.5 Summary 11
3 Which Tcl? **11**
3.1 Introduction 11
3.2 The three origins 11
3.3 Who answers? 12
3.4 Tk for Tcl 9.1 13
3.5 A source build that loads Debian's library 13
3.6 tclsh and wish in the PATH 14
3.7 The shebang line 14
3.8 How to recognize a Tcl 9 14
3.9 Summary 15
4 Paths, environment, XDG **15**
4.1 Introduction 15
4.2 HOME and the tilde 15
4.3 Environment variables 16
4.4 XDG Base Directory 16
4.5 User directories: Documents and Desktop 17
4.6 File system 17
4.7 Summary 18
5 Search paths **18**
5.1 Introduction 18
5.2 auto_path of the distribution Tcl 18
5.3 auto_path of the Debian packages for Tcl 9 19
5.4 auto_path of a source build 19
5.5 sqlite3 under Debian's Tcl 9 20
5.5.1 With 3.53.4-2 or older 20
5.5.2 Own build and package 21
5.6 TCLLIBPATH 21
5.7 tm paths 21
5.8 Which copy wins 22
5.9 Which package was loaded? 23
5.10 Summary 23
6 Encoding and locale **23**

6.1 Introduction 23
6.2 encoding system follows the locale 24
6.3 Files: always with an encoding 24
6.4 Scripts: the guard under Tcl 8.6 25
6.5 Terminal and exec 25
6.6 msgcat and language 26
6.7 Summary 26
7 exec, xdg-open, processes 26
7.1 Introduction 26
7.2 Finding a program: auto_execok 26
7.3 Telling exec errors apart 27
7.4 In the background and via channels 27
7.4.1 The pitfall with your own selection 28
7.5 xdg-open: opening a file or address 28
7.6 Process ID and signals 29
7.7 Summary 29
8 X11, window managers and Wayland 29
8.1 Introduction 29
8.2 DISPLAY and starting without a display 30
8.3 Window class and WM_CLASS 30
8.4 What the window manager provides 31
8.5 Detecting a window manager 32
8.6 Wayland 32
8.6.1 XWayland under Weston 33
8.7 Summary 33
9 Fonts, scaling, themes 34
9.1 Introduction 34
9.2 Fonts: Xft and fontconfig 34
9.3 tk scaling and DPI 34
9.3.1 Where Tk 9 gets the scaling from 35
9.4 Themes 36
9.5 X resources 36
9.6 Summary 37
10 Mouse, keyboard, clipboard 37
10.1 Introduction 37
10.2 The mouse wheel 37
10.3 Modifiers and keys 38
10.4 Two clipboards 39
10.4.1 Virtual events 40
10.5 Summary 41
11 Native dialogs and desktop integration 41
11.1 Introduction 41
11.2 The standard dialogs are Tk dialogs 41
11.3 The .desktop file 42
11.4 Window icon 43
11.5 Notifications 43
11.6 Icon in the panel 43

11.7 Printing 44
11.8 Summary 44
12 Xfce from Tcl 44
12.1 Introduction 44
12.2 Detecting Xfce 45
12.3 xfconf: reading settings 45
12.4 xfconf: writing settings 46
12.5 Preferred applications: exo-open 46
12.6 Notifications 46
12.7 Window manager, panel, session 46
12.8 Summary 47
13 GUI tests with Xvfb 47
13.1 Introduction 47
13.2 Xvfb and xvfb-run 47
13.2.1 Own server instead of xvfb-run 48
13.3 Screen, DPI and fonts 48
13.4 Exit status and abort 48
13.5 tclsh or wish 49
13.6 Summary 49
14 What the test sets up itself 49
14.1 Introduction 49
14.2 Overview 50
14.3 Mapping: update first 50
14.4 Generating keys: establishing focus 50
14.5 Grab 51
14.6 Position of dialogs 51
14.6.1 The frame comes late 52
14.6.2 openbox places centered dialogs itself 52
14.7 Tests that need a window manager 53
14.8 Summary 53
15 Building Tk: dependencies 53
15.1 Introduction 54
15.2 What Debian needs to build Tk 54
15.3 TK_LIBS and TCL_LIBS 54
15.4 What is really linked 55
15.5 tclConfig.sh on Debian 55
15.6 Threads and shared 55
15.7 Two pitfalls when building 56
15.8 Summary 56
16 Distributing 56
16.1 Introduction 56
16.2 Which Tcl does the target machine have? 56
16.3 An image with zipfs mkimg 57
16.4 Zipkit: static base 58
16.5 Desktop: launcher and icon 58
16.6 Summary 58
17 FAQ 59

17.1 Platform and interpreter 59
17.2 Encoding 59
17.3 Processes 59
17.4 User interface 60
17.5 Tests 60
17.6 Building and distributing 60
18 Pitfalls 61
18.1 Platform and interpreter 61
18.2 Paths and search paths 61
18.3 Encoding 62
18.4 Processes 62
18.5 X11 and window managers 62
18.6 Font, scaling, themes 62
18.7 Mouse, keyboard, clipboard 63
18.8 Desktop 63
18.9 Tests 63
18.10 Building and distributing 63
19 CLI 63
19.1 At runtime 64
19.2 Shell 64
19.3 Check script 64
19.4 More example scripts 65
19.5 Howto series 66
20 Glossary 66
20.1 Symbols 66
20.2 A 67
20.3 C 67
20.4 D 67
20.5 E 67
20.6 F 67
20.7 L 68
20.8 M 68
20.9 O 68
20.10 P 68
20.11 R 68
20.12 S 68
20.13 T 68
20.14 W 69
20.15 X 69
20.16 Z 70
21 Error messages 70
21.1 no display name and no $DISPLAY environment variable 70
21.2 couldn't connect to display 70
21.3 couldn't execute … no such file or directory 70
21.4 child process exited abnormally 70
21.5 child killed: software termination signal 70
21.6 (text on stderr) 70

21.7 invalid or incomplete multibyte or wide character 70
21.8 unknown or ambiguous subcommand "home" 71
21.9 unknown or ambiguous subcommand "systray" 71
21.10 bad button number "8" 71
21.11 bad event type or keysym "Super" 71
21.12 CLIPBOARD selection doesn't exist or form "STRING" not defined 71
21.13 Error: target STRING not available 71
21.14 Unable to obtain the list of printers 71
21.15 Failed to execute child process "dbus-launch" 71
21.16 Verbindung ist gescheitert 72
21.17 SKIP: serie-xfce braucht XFCE 72
21.18 WARNING: unrecognized options 72
21.19 file format not recognized 72
21.20 XInput2.h missing 72
21.21 this extension is compiled for Tcl 8.6 72
21.22 Keine Berechtigung (permission denied) when installing to /usr/bin 72
22 Sources 72
22.1 Introduction 73
22.2 Own runs 73
22.2.1 The author's howto series 73
22.2.2 Container check 73
22.2.3 Check on lxpro 73
22.2.4 What was missing in the container 74
22.3 Tcl and Tk 74
22.4 Debian 75
22.5 Specifications 75
22.6 Companion books in the same series 75
22.7 Contradictions between the sources 76
22.8 Trademarks 76
23 Test environment 76
23.1 Introduction 76
23.2 Machines and versions 76
23.3 Check scripts 77
23.4 Not yet tested 77
24 License 77

1 What this book is

1.1 Introduction

Tcl/Tk is at home on Linux: the distributions ship it as a package, `wish` draws with the X11 windowing system, and most Tcl examples on the web silently assume Linux. Still, a Tcl program on a Linux workstation behaves differently than on macOS or Windows in many places: several interpreters sit side by side, paths follow XDG, the encoding comes from the locale, a window manager decides on focus and size, and there are two clipboards.

This book covers these particularities from the point of view of Tcl/Tk development: the platform value, which Tcl is currently running, paths and search paths of the distribution, encoding, `exec`, X11 and window managers, fonts and scaling, mouse wheel and clipboard, integration into the desktop, Xfce, GUI tests under Xvfb, the build dependencies of Tk, and distribution.

It is a workshop book. The chapters show procedures and commands that have been tried on Linux, with the output they produce, and with the places where something did not go as expected. It assumes general Tcl/Tk knowledge. It is not a Linux manual.

1.2 What the examples work with

The examples use Tcl 8.6 from the distribution package and Tcl/Tk 9.0.4, self-built under `/opt/tcl9`, on Debian and Ubuntu with X11. Where 8.6 and 9.0 differ, the text says so, likewise when a value only holds with one window manager or only under Xfce.

The basis is five howto series by the author. `serie-linux`, `serie-xvfb` and `serie-xfce` are published, in the repository `tcltk-manuals` under Serien/; `serie-tcltk-bauen` and `serie-tcl9-app` are not. Notes like "(`serie-linux`, howto 08)" name the script a value was checked with. Each series has a `README.md` with the checked values, a `TUTORIAL.md`, a short reference `howto/00-referenz.md`, and for each howto an explanation `NN-*.md` with a runnable demo `NN-*.tcl`. The demos check their own claims and report OK, SKIP or FAIL.

Series	Content	Chapters here
serie-linux	Paths, XDG, environment, exec, encoding, file system, tcl_platform, dialogs	2, 4, 6, 7, 11
serie-xvfb	Xvfb, window managers, focus, iconify, grab, tests without decorator (20 howtos)	8, 13, 14
serie-xfce	Xfce 4.20 from Tcl: xfconf, DBus, Thunar, exo-open, notify-send	11, 12
serie-tcltk-bauen	Build dependencies of Tk on Debian, TK_LIBS, ldd	2, 3, 5, 15
serie-tcl9-app	Programs with Tcl 9 and zipfs	16

The book does not take over the howtos piece by piece. It arranges them by topic, explains the context, and keeps the lines of code that matter. When the series run as a whole, `SERIES_AUTO=1` is set; then no demo opens a dialog, a browser or a notification window.

To check a claim yourself, use the scripts in `beispiele/` next to the book folder, above all

messprobe-linux.tcl, which prints all values of the tables in this book.

Machines, exact versions and what has not been tested yet are listed in Appendix F: Test environment. Values from the Ubuntu container are called "container check" in the text; there, Xvfb runs without a window manager, without a desktop session and without Xfce.

1.3 How to read the book

Part 1 (chapters 1 to 7) explains the platform: `tcl_platform`, which Tcl, paths and XDG, search paths, encoding and locale, `exec`. Part 2 (chapters 8 to 12) goes to the user interface: X11 and window managers, fonts and scaling, mouse, keyboard and clipboard, dialogs and desktop, Xfce. Part 3 (chapters 13 to 17) covers tests under Xvfb, the build dependencies of Tk, distribution, and frequently asked questions.

The appendices are for reference: pitfalls (A), commands (B), glossary (C), error messages (D), sources (E) and the test environment (F). Appendix A is worth keeping open alongside while reading.

Other topics are in separate books of the same series: Debian as an operating system in *Debian unter Linux* (this book describes the interpreter on the distribution, that one the distribution), the sister books *Tcl/Tk unter macOS* and *Tcl/Tk unter Windows*, building Tcl/Tk yourself in *Tcl/Tk 9.0 bauen* and *Tcl/Tk 9.1 bauen*, kits, zipfs, `.deb` and `.rpm` in *Tcl-Anwendungen ausliefern* (in the text *Ausliefern* for short), C extensions in *Tcl-Erweiterungen*, packages and modules in *Pakete und Module*.

Next chapter: tcl_platform and the distribution

2 tcl_platform and the distribution

2.1 Introduction

tcl_platform tells which operating system Tcl sees. On Linux and on macOS, `platform` is the same: `unix`. The distinction is made by `os`. Which distribution is running, `tcl_platform` does not tell. For that there is /etc/os-release.

2.2 The keys

```
set ::tcl_platform(platform)   ;# unix
set ::tcl_platform(os)         ;# Linux
set ::tcl_platform(osVersion)  ;# kernel, not distribution
set ::tcl_platform(machine)    ;# x86_64
set ::tcl_platform(user)       ;# user of the process
```

Values from `serie-linux` (Debian forky, Tcl 8.6.18, howto 10) and from the container check (– = not collected there):

Key	Debian forky, 8.6.18	Container, 8.6.14	Container, 9.0.4
platform	unix	unix	unix
os	Linux	Linux	Linux

Key	Debian forky, 8.6.18	Container, 8.6.14	Container, 9.0.4
osVersion	7.2.14+deb14-amd64	6.18.44-fc-v139	6.18.44-fc-v139
machine	x86_64	x86_64	x86_64
wordSize	8	8	8
pointerSize	–	8	8
byteOrder	–	littleEndian	littleEndian
threaded	1 (8.6)	1	key missing

Linux code therefore checks:

```
if {$::tcl_platform(os) eq "Linux"} { ... }
```

`platform eq "unix"` also matches macOS (`os` is `Darwin` there) and the BSDs. For "X11 or not", `tcl_platform` is the wrong witness anyway; that is told by `tk windowingsystem`, see X11, window managers and Wayland.

2.2.1 threaded under Tcl 9

Tcl 9 is always built with threads. The key threaded is therefore missing. Code that reads it needs `info exists`:

```
set threads [expr {![info exists ::tcl_platform(threaded)]
                   || $::tcl_platform(threaded)}]
```

`serie-tcltk-bauen` (howto 05) confirms this for 9.0.4 on Debian: `TCL_THREADS=1` in the `tclConfig.sh` of 8.6, no key under 9.0, and `--enable-threads` is an unknown switch there.

2.2.2 osVersion is the kernel

`osVersion` is the output of `uname -r` and changes with every kernel update. On the Debian workstation it was `7.2.14+deb14-amd64` at the last check (`serie-linux`, howto 10, still lists `7.1.9+deb14-amd64`), in the container `6.18.44-fc-v139`. The second value is the kernel of the host, not that of Ubuntu: a container does not bring its own kernel. A version check for "Debian 13" or "Ubuntu 24.04" via `osVersion` therefore always goes wrong.

2.2.3 user is not USER

`tcl_platform(user)` is the name for the effective user ID of the process. `$env(USER)` is inherited and can be missing or say something else, for example after `sudo` or in a sandbox. In the container, started with `env -i`: `tcl_platform(user)` is `root`, `env(USER)` does not exist. `serie-linux` (howto 03) therefore fails in the container with "env USER gesetzt", but not on the workstation.

```
set wer $::tcl_platform(user)             ;# always there
set wer2 [expr {[info exists ::env(USER)] ? $::env(USER) : $wer}]
```

2.3 Distribution: /etc/os-release

The distribution is in `/etc/os-release`. The file consists of lines `KEY=value`, values often in quotation marks. It is shell syntax, but for the usual keys a simple reader is enough:

```
proc osRelease {} {
    set d [dict create]
    foreach datei {/etc/os-release /usr/lib/os-release} {
        if {![file readable $datei]} continue
        set ch [open $datei]
        fconfigure $ch -encoding utf-8
        foreach zeile [split [read $ch] \n] {
            if {[regexp {^([A-Z_]+)=(.*)$} $zeile -> k v]} {
                dict set d $k [string trim $v {"'}]
            }
        }
        close $ch
        break
    }
    return $d
}
set r [osRelease]
puts "[dict get $r ID] [dict getdef $r VERSION_CODENAME -]"
```

`dict getdef` exists from Tcl 8.7 on; under 8.6 check `dict exists` first. Result (– = not collected):

Key	Debian workstation	Container
ID	debian	ubuntu
ID_LIKE	–	debian
VERSION_ID	missing	24.04
VERSION_CODENAME	forky	noble
PRETTY_NAME	Debian GNU/Linux forky/sid	Ubuntu 24.04.5 LTS

The Debian column comes from `serie-linux` (howto 06: `ID=debian`, `VERSION_CODENAME=forky`). Ubuntu carries `ID_LIKE=debian`, as the Debian book writes in the chapter *Debian erkennen*. Debian Testing does not set `VERSION_ID` (workstation, forky/sid). Code therefore does not rely on it and takes `VERSION_CODENAME`. To check for "Debian or derived", read `ID` **and** `ID_LIKE`.

```
set familie [concat [dict getdef $r ID ""] [dict getdef $r ID_LIKE ""]]
if {"debian" in $familie} { ... }
```

Incidentally, in the container `/etc/debian_version` returns `trixie/sid`, although the system is Ubuntu. Ubuntu takes the file over from Debian. It is no use for telling Debian from Ubuntu.

2.4 The running process: /proc/self

What Windows reads from the registry and macOS from `sysctl` lies in the /proc file system on Linux. `serie-linux` (howto 06):

```
file readlink /proc/self/exe      ;# e.g. /usr/bin/tclsh8.6
set ch [open /proc/self/comm]
set name [string trim [read $ch]]
close $ch
```

`/proc/self/exe` is the resolved path of the interpreter, even if it was started via a symlink. `info nameofexecutable`, on the other hand, returns the path under which the interpreter was called. The question "which Tcl is running here" is the subject of the next chapter.

2.5 Summary

- `platform` is `unix` on Linux and macOS. `os` distinguishes: `Linux`.
- `osVersion` is the kernel, in the container that of the host.
- `threaded` is missing under Tcl 9. Use `info exists` before reading.
- `tcl_platform(user)` is reliable, `$env(USER)` is not.
- Distribution from `/etc/os-release`, family via `ID` and `ID_LIKE`. `/etc/debian_version` also exists on Ubuntu.

Next chapter: Which Tcl?

3 Which Tcl?

3.1 Introduction

On a Linux workstation there are often several Tcls side by side: the distribution's under `/usr` and a self-built one under `/opt`. Both are called `tclsh` with a version number. Which one answers is decided by the `PATH` or the full path. This chapter shows how to recognize the running Tcl and how the two kinds differ.

3.2 The three origins

Origin	Location	Invocation	checked with
Distribution package Tcl 8.6	/usr/bin	tclsh8.6, wish8.6, often tclsh	Debian forky 8.6.18, Ubuntu 24.04 8.6.14
Distribution package Tcl 9.0	/usr/bin	tclsh9.0, wish9.0	Debian forky 9.0.4 (serie-tcltk-bauen)
Distribution package Tcl 9.1	/usr/bin	tclsh9.1, wish9.1	Debian forky and unstable 9.1.0
Source build	/opt/tcl9	/opt/tcl9/bin/tclsh9.0, wish9.0	Container 9.0.4

Which packages the Debian branches carry, among them tcl9.0 (source packages, as of 2026-11-02, `qa.debian.org/madison.php`):

Package	Stable (trixie)	Testing (forky)	unstable (sid)
tcl8.6	8.6.16	8.6.18	8.6.18

Package	Stable (trixie)	Testing (forky)	unstable (sid)
tcl9.0	9.0.1	9.0.4	9.0.4
tcl9.1	–	9.1.0	9.1.0

For each `tcl…` there is the matching `tk…` in the same version. The maintainer of the Tcl packages announced in November 2026 that Tcl/Tk 9.0 will be dropped for forky; 9.1 would then remain alongside 8.6. 8.6 stays the default: Debian builds packages with a C part only against 8.6, as long as the `tcltk-defaults` packages say nothing else. The Debian book, chapter *Tcl unter Linux*, describes the situation on the author's machines: on the Debian workstation the packages for 8.6.18, 9.0.4 and 9.1.0 under `/usr` and a self-built 9.1.0 under `/opt/tcl91`, on a second machine a self-built 9.0.4 under `/opt/tcl9`. Ubuntu 24.04 in the container has only 8.6.14 as a package.

The rules from the Debian book apply here unchanged: never install your own Tcl into `/usr`, but into `/opt`. The build itself is described in *Tcl/Tk 9.0 bauen* and *Tcl/Tk 9.1 bauen*, chapter *Mehrere Installationen*.

3.3 Who answers?

In the shell:

```
type -a tclsh tclsh8.6 tclsh9.0 wish9.0
ls -l /usr/bin/tclsh /usr/bin/wish
dpkg -S /usr/bin/tclsh8.6          # Debian and Ubuntu only
```

In the container, `/usr/bin/tclsh` is a symlink to `tclsh8.6`, `/usr/bin/wish` one to `wish8.6`. `update-alternatives --display tclsh` reports "no alternatives for tclsh" there: the link belongs to the package `tcl`, not to the alternatives system.

In the interpreter:

```
puts [info nameofexecutable]    ;# path of the invocation
puts [info patchlevel]
puts $tcl_library
puts [tcl::pkgconfig get libdir,runtime]
```

Container check:

Value	tclsh8.6 (Ubuntu)	tclsh9.0 (/opt/tcl9)
info patchlevel	8.6.14	9.0.4
$tcl_library	/usr/share/tcltk/tcl8.6	//zipfs:/lib/tcl/tcl_library
libdir,runtime	/usr/lib/x86_64-linux-gnu	/opt/tcl9/lib
scriptdir,runtime	/usr/share/tcltk/tcl8.6	/opt/tcl9/lib/tcl9.0
zipfs mount	command missing	//zipfs:/lib/tcl on libtcl9.0.so

Two things stand out. Debian and Ubuntu put the Tcl library into `/usr/share/tcltk`, not into `/usr/lib/tcl8.6` like a source build of 8.6. And a source build of Tcl 9 attaches the script library as a ZIP to `libtcl9.0.so`. `$tcl_library` then points into zipfs. The folder `/opt/tcl9/lib/tcl9.0` from `scriptdir,runtime` does not exist in the container; it is a

leftover of the build default.

The Debian packages for Tcl 9 store the library without zipfs. On the Debian workstation, `tclsh9.0` reports the folder `/usr/lib/tcl9.0`, `tclsh9.1` the folder `/usr/lib/tcl9.1`.

3.4 Tk for Tcl 9.1

Debian splits Tcl and Tk into separate packages. If you install only `tcl9.1`, you have no `wish9.1`, and `package require Tk` in `tclsh9.1` still reports a version. On the Debian workstation, without the package `tk9.1`:

```
% package require Tk
9.0.4
```

`tclsh9.1` finds the index of Debian's Tk 9.0 under `/usr/lib/tcltk/x86_64-linux-gnu/tk9.0` via `auto_path` and loads it. That works because Tk 9.0 binds to Tcl via the stub table; the release notes of Tk 9.1.0 explicitly name Tcl 9.0 or 9.1 as the basis. But it is not Tk 9.1: the new features of 9.1 are missing. Remedy:

```
sudo apt install tk9.1
```

After that, `package require Tk` in `tclsh9.1` returns version 9.1.0, and `wish9.1` is there.

3.5 A source build that loads Debian's library

A Tcl under `/opt` is only separate if it also loads its own `libtcl`. The file has the same name in Debian and in the source build, `libtcl9.0.so`. If the loader finds Debian's file first, the program from `/opt` runs with Debian's library. On the Debian workstation with a BAWT build 9.0.1 under `/opt/Tcl`:

Invocation	info patchlevel	$tcl_library
/opt/Tcl/bin/tclsh9.0	9.0.4	/usr/lib/tcl9.0
LD_LIBRARY_PATH=/opt/Tcl/lib /opt/Tcl/bin/tclsh9.0	9.0.1	//zipfs:/lib/tcl/tcl_library

Without `LD_LIBRARY_PATH`, the BAWT program reports Debian's version and Debian's script library. It then also sees Debian's packages first. This can be checked with `ldd`:

```
ldd /opt/Tcl/bin/tclsh9.0 | grep libtcl
```

If the line points to `/usr/lib/x86_64-linux-gnu`, the program needs a small launcher file, for example `~/bin/tclsh9b`:

```
#!/bin/sh
LD_LIBRARY_PATH=/opt/Tcl/lib${LD_LIBRARY_PATH:+:$LD_LIBRARY_PATH} exec
    /opt/Tcl/bin/tclsh9.0 "$@"
```

An alias is not enough: it only applies in the interactive shell, not in scripts and makefiles. The source build into `/opt/tcl9` in the container needs no launcher file: `readelf -d /opt/tcl9/bin/tclsh9.0` shows `RUNPATH /opt/tcl9/lib`, and the program finds its own library before that of the system. In the BAWT build on the Debian workstation no such entry takes effect; what is there is shown by `readelf -d` on the machine itself.

3.6 tclsh and wish in the PATH

A source build into `/opt/tcl9` puts `tclsh9.0` and `wish9.0` into `/opt/tcl9/bin`. In the container there was additionally a link `tclsh -> tclsh9.0` there, but no link `wish`. `make install` does not create this link: the target `install-binaries` in `unix/Makefile.in` of 9.0.4 installs only `tclsh9.0`. So the link was created by hand, as *Tcl/Tk 9.0 bauen* suggests in the chapter *Mehrere Installationen*. Consequence: with `/opt/tcl9/bin` at the front of the PATH, `tclsh` starts Tcl 9, but `wish` still starts the `wish8.6` from `/usr/bin`.

Therefore always invoke with the version number. For 9.1 alongside 9.0, the Debian book advises not to put both prefixes into the PATH, but to use the full path.

3.7 The shebang line

An executable script names its interpreter in the first line. On Linux there are three common forms:

```
#!/usr/bin/tclsh8.6
#!/usr/bin/env tclsh9.0
#!/opt/tcl9/bin/wish9.0
```

`/usr/bin/env` searches the PATH. That makes the script portable, but dependent on the session: a program from the start menu sees a different PATH than a terminal (see Native dialogs and desktop integration). The full path is fixed, but tied to one machine.

The Linux kernel passes everything after the interpreter path as **one** argument. `#!/usr/bin/env tclsh9.0 -encoding utf-8` therefore looks for a program named `tclsh9.0 -encoding utf-8`. For options there is `env -S` from GNU coreutils; check it on the target machine first.

The old form for systems on which the path to `wish` is unknown uses the fact that Tcl reads a backslash at the end of a comment as a continuation, but the shell does not:

```
#!/bin/sh
# Only the shell reads the next line, for Tcl it is a comment \
exec wish9.0 "$0" "$@"
```

`serie-tcl9-app` (howto 02) uses `#!/usr/bin/env tclsh9.0` for Linux.

3.8 How to recognize a Tcl 9

```
if {[package vsatisfies [info patchlevel] 9-]} { ... }
```

Without a version comparison, Tcl 9 gives itself away by three things (checked in the container):

- `$tcl_library` starts with `//zipfs:/` (in the source build).
- `file volumes` returns `//zipfs:/ /`, under 8.6 only `/`.
- `file attributes /tmp -permissions` returns `0o41777`, under 8.6 `041777`.

The third affects code that compares the permission value as a string. `expr` reads numbers with `0o` under both versions. Code that compares permissions therefore calculates with the numeric value:

```
set r [file attributes $datei -permissions]
expr {$r & 0o777}        ;# same under 8.6 and 9
```

3.9 Summary

- Debian Stable: 8.6.16 and package `tcl9.0` 9.0.1. Testing and unstable: 8.6.18, `tcl9.0` 9.0.4 and `tcl9.1` 9.1.0. Announced: 9.0 is dropped for forky, 9.1 remains. Your own Tcl into `/opt`, never into `/usr`.
- Debian puts `$tcl_library` into `/usr/share/tcltk/tcl8.6` and `/usr/lib/tcl9.x`. A source build of Tcl 9 puts it into zipfs on `libtcl9.0.so`.
- Without the package `tk9.1`, `tclsh9.1` loads Debian's Tk 9.0.
- A program under `/opt` can load Debian's `libtcl`. `ldd` checks this, a launcher file with LD_LIBRARY_PATH fixes it.
- Always invoke with the version number. `wish` without a number is usually 8.6.
- A shebang with `env` follows the PATH of the session.
- `file volumes` and the format of permissions differ between 8.6 and 9.

Next chapter: Paths, environment, XDG

4 Paths, environment, XDG

4.1 Introduction

On Linux every process has a HOME, paths are separated by the slash, and there is only one root directory. Where a program stores its settings is governed by the XDG Base Directory Specification: not as a dot file directly in $HOME, but in `~/.config/<program>/`. This chapter shows the variables, the defaults when they are missing, and the break in tilde handling between Tcl 8.6 and 9.

4.2 HOME and the tilde

`$env(HOME)` is set on Linux, unlike often on Windows (`serie-linux`, howtos 02 and 03). Paths are built with `file join`:

```
set konfig [file join $env(HOME) .config meineapp]
```

With the tilde, 8.6 and 9 part ways. Tcl 8.6 replaces a leading ~ in every path with HOME. Tcl 9 no longer does that (TIP 602): ~ is an ordinary file name there. In return there are two new commands. Container check, working directory `/tmp`, HOME=`/root`:

Expression	Tcl 8.6.14	Tcl 9.0.4 and 9.1.0
file normalize ~ (started in /tmp)	/root	/tmp/~
file exists ~	1	0
file home	error, unknown	/root
file tildeexpand ~/x	error, unknown	/root/x

On the Debian workstation with the Debian packages, started in the home directory, the same

picture: `tclsh` 8.6.18 returns the home directory for `file normalize ~` and 1 for `file exists ~`, `tclsh9.0` 9.0.4 and `tclsh9.1` 9.1.0 return `<home directory>/~` and 0.

So `file normalize ~` only works as a way to the home directory under 8.6. Under Tcl 9 the same expression returns a path in the current directory. `serie-linux` shows both versions in howto 02; the demo `02-pfade.tcl` checks "Tilde = HOME" under 8.6 and `file home` and `file tildeexpand` under 9.

Code for both versions uses `$env(HOME)` and no tilde:

```
proc heim {} {
    if {[info exists ::env(HOME)] && $::env(HOME) ne ""} {
        return $::env(HOME)
    }
    return [file home]          ;# Tcl 9 only
}
```

A tilde that comes from outside, for example from a configuration file, is only resolved under Tcl 9 by `file tildeexpand`. Under 8.6 every file command does that by itself.

4.3 Environment variables

`serie-linux` (howto 03) reads the usual variables:

```
$env(HOME) $env(USER) $env(SHELL) $env(PATH)
$env(LANG) $env(DISPLAY)
```

Which of them are set depends on the session. In a graphical session, `DISPLAY`, `XDG_CURRENT_DESKTOP`, `XDG_SESSION_TYPE` and `XDG_RUNTIME_DIR` are available. In a cron run, under `sudo` or with `env -i` they are missing. `HOSTNAME` and `TMPDIR` are often missing in the session too. For the host name there is `info hostname`. The temporary directory is then `/tmp`.

Access with a default is robust:

```
proc envOder {name vorgabe} {
    if {[info exists ::env($name)] && $::env($name) ne ""} {
        return $::env($name)
    }
    return $vorgabe
}
set tmp [envOder TMPDIR /tmp]
```

Tcl 9 also has `file tempdir` for this. It creates a new, empty directory and returns its path, in the container `/tmp/tcl_KZSgAw`. The program removes the directory again itself.

4.4 XDG Base Directory

The specification defines four locations for user data. Each variable may be missing; then the default applies. `serie-linux` (howto 04):

Variable	Default	Content
XDG_CONFIG_HOME	~/.config	Settings

Variable	Default	Content
XDG_DATA_HOME	~/.local/share	Data that should persist
XDG_CACHE_HOME	~/.cache	Disposable data
XDG_STATE_HOME	~/.local/state	History, last window size
XDG_RUNTIME_DIR	none	Sockets, lock files; /run/user/UID

On the author's workstation `XDG_DATA_HOME` is often not set, `XDG_RUNTIME_DIR` is `/run/user/UID`. In the container with `env -i` none of these variables is set. `XDG_STATE_HOME` is not checked by the series; the default comes from the specification.

A program puts its files into its own subdirectory:

```
proc xdgOrdner {var vorgabe app} {
    set basis ""
    if {[info exists ::env($var)]} { set basis $::env($var) }
    if {$basis eq "" || [file pathtype $basis] ne "absolute"} {
        set basis [file join [heim] {*}$vorgabe]
    }
    return [file join $basis $app]
}
set konfig [xdgOrdner XDG_CONFIG_HOME .config meineapp]
file mkdir $konfig
```

According to the specification, a relative value is invalid and is ignored. `beispiele/xdg-ordner.tcl` contains the four variants and ran in the container under 8.6.14 and 9.0.4 with the same result, also with `XDG_CONFIG_HOME=relativ`: `/root/.config/meineapp`.

The Windows equivalent is `APPDATA`, the macOS equivalent `~/Library/Application Support`. A program for all three first asks `tcl_platform(os)` and then the respective source.

4.5 User directories: Documents and Desktop

The names of the visible folders in the home directory are **localized**. On a German desktop the folder is called `Dokumente`, not `Documents`, and the desktop `Schreibtisch`. The mapping is in ~/.config/user-dirs.dirs; the tool `xdg-user-dir` reads it (`serie-linux`, howto 05):

```
exec {*}[auto_execok xdg-user-dir] DOCUMENTS
exec {*}[auto_execok xdg-user-dir] DESKTOP
exec {*}[auto_execok xdg-user-dir] DOWNLOAD
```

`xdg-user-dir` belongs to the Debian package `xdg-user-dirs`. It is missing in the container; howto 05 skips there. `beispiele/xdg-ordner.tcl` falls back to `HOME` in this case. Hard-coding a folder like `~/Documents` is the most common mistake of this section.

4.6 File system

Unix attributes, no drive letters (`serie-linux`, howto 09):

```
file volumes                          ;# /
file attributes $f                    ;# -group -owner -permissions
file attributes $f -permissions 00644
```

Under Tcl 9, `file volumes` additionally returns `//zipfs:/` (container check: `//zipfs:/ /`). Code that takes `[lindex [file volumes] 0]` for the root gets zipfs under 9. `/tmp` has the sticky bit, permissions `1777`. The format of the permission value differs between 8.6 and 9, see Which Tcl?.

`file nativename` returns the same path on Linux. Case matters: `Daten.txt` and `daten.txt` are two files. This sets Linux apart from macOS (APFS in the default setting) and from Windows.

4.7 Summary

- HOME is set. Paths with `file join`.
- Tilde: resolved everywhere under 8.6, no longer under 9. `file normalize ~` returns a path in the current directory under 9. For both versions, `$env(HOME)`.
- XDG: variable or default, own subdirectory, ignore relative values.
- User directories are localized, via `xdg-user-dir`.
- `file volumes` also contains `//zipfs:/` under 9.

Next chapter: Search paths

5 Search paths

5.1 Introduction

`package require` searches in auto_path, modules (`.tm`) in the tm path. Every Tcl assembles both lists at startup from built-in values and environment variables. Debian and Ubuntu change the built-in values with their own patches. A self-built Tcl under `/opt` does not know them. Anyone who has both side by side must know who sees which packages.

The basics of `package`, `pkgIndex.tcl` and `.tm` are in the book *Pakete und Module*. This chapter covers only the Linux values.

5.2 auto_path of the distribution Tcl

Container check, `tclsh8.6` from Ubuntu 24.04, with `env -i`, without `TCLLIBPATH`:

```
/usr/share/tcltk/tcl8.6
/usr/share/tcltk
/usr/lib
/usr/local/lib/tcltk
/usr/local/share/tcltk
/usr/lib/tcltk/x86_64-linux-gnu
/usr/lib/tcltk
/usr/lib/tcltk/tcl8.6
```

The list follows the Debian Tcl policy: architecture-independent packages under /usr/share/tcltk, packages with a C part under `/usr/lib/tcltk/<multiarch>`, local packages under `/usr/local/share/tcltk` and `/usr/local/lib/tcltk`. `tcl_pkgPath` names the same

package directories in a different order, but not the library directory of the version. That one is only in `auto_path`, at the front for 8.6: `/usr/share/tcltk/tcl8.6`. Correspondingly, Debian's `tclsh9.0` lacks `/usr/lib/tcl9.0` in `tcl_pkgPath`, and `tclsh9.1` lacks `/usr/lib/tcl9.1`. Debian unstable with 8.6.18 returns the same list in the same order.

Packaged libraries live there with the version number in the directory name. In the container, `struct::list` loads from `/usr/share/tcltk/tcllib1.21/struct/list.tcl`.

Your own packages for the system Tcl belong in `/usr/local/share/tcltk` (pure Tcl) or `/usr/local/lib/tcltk` (with `.so`). There the system Tcl finds them without further configuration, and `apt` does not touch them.

5.3 auto_path of the Debian packages for Tcl 9

On Debian unstable, `tclsh9.0` from the package `tcl9.0`, without the entries from TCLLIBPATH:

```
/usr/lib/tcl9.0
/usr/lib
/usr/local/lib/tcltk
/usr/local/share/tcltk
/usr/lib/tcltk/x86_64-linux-gnu
/usr/lib/tcltk
/usr/share/tcltk
/usr/lib/tcltk/tcl9.0          (fehlt)
```

`tclsh9.1` reports the same list with `9.1` instead of `9.0`. Unlike a source build, Debian's Tcl 9 therefore sees **all** package directories of the distribution, including those with libraries only for 8.6. Whether a package loads under Tcl 9 is then decided by its `pkgIndex.tcl` alone. Packages like tdom, tcltls, Thread or Img put two libraries side by side, `libtdom0.9.6.so` for 8.6 and `libtcl9tdom0.9.6.so` for 9, and their index chooses by version. Where the second file is missing, `package require` fails under Tcl 9 (section *sqlite3 under Debian's Tcl 9*).

In the list of tm paths, the packages for Tcl 9 show two directories as a single entry:

```
/usr/lib/tcltk /usr/share/tcltk/tcl9/site-tcl
/usr/lib/tcltk /usr/share/tcltk/tcl9/9.0
```

Tcl 9 does not find modules under `/usr/share/tcltk/tcl9` this way. On the Debian unstable system checked, these directories do not exist, so nothing is missing at present.

5.4 auto_path of a source build

Container check, `/opt/tcl9/bin/tclsh9.0`, `env -i`:

```
//zipfs:/lib/tcl/tcl_library
//zipfs:/lib/tcl
/opt/tcl9/lib
```

That is all. `/usr/share/tcltk` is not in it. A Tcl 9 under `/opt` sees not a single package of the distribution, neither Tk extensions nor Tcllib. That is intentional: a package for 8.6 with a C part does not load under Tcl 9 anyway. Pure Tcl packages often run under both versions, but must be added explicitly.

In the container, `/opt/tcl9/lib` brought its own versions: Tcllib 1.9 (`package versions`

`struct::list`), Thread 3.0.6, sqlite3 3.53.0. In total, after a `package require` for a missing package, `tclsh9.0` knew 628 package names, `tclsh8.6` 584.

What `package require` returns on Debian forky/sid (Debian packages, without `TCLLIBPATH`); `serie-tcltk-bauen`, howto 10, still lists Thread 3.0.6 and tdbc 1.1.14:

Package	Tcl 8.6.18	Tcl 9.0.4	Tcl 9.1.0
Thread	3.0.7	3.0.7	3.0.7
Itcl	4.3.8	4.3.8	4.3.8
sqlite3	3.53.4	missing	missing
tdbc	1.1.15	1.1.15	1.1.15

Thread comes from the package `tcl-thread` 1:3.0.7-2, tdbc from `/usr/lib/tcltk/x86_64-linux-gnu/tdbc1.1.15`. sqlite3 fails under 9.0 and 9.1 with `this extension is compiled for Tcl 8.6` (section *sqlite3 under Debian's Tcl 9*).

A source build takes sqlite from the `pkgs/` directory of the Tcl tree; the Debian package `tcl9.0` lacked it there.

5.5 sqlite3 under Debian's Tcl 9

The package libsqlite3-tcl contains, up to 3.53.4-2, only one library, built for Tcl 8.6. Under `tclsh9.0` and `tclsh9.1`, `package require sqlite3` then fails:

```
% package require sqlite3
this extension is compiled for Tcl 8.6
```

Since 3.54.1-1 (unstable, November 2026) the package ships a second library for Tcl 9; then there is nothing to do:

```
/usr/lib/tcltk/x86_64-linux-gnu/sqlite3/
    libsqlite3.54.1.so        for 8.6
    libtcl9sqlite3.54.1.so    for 9.0 and 9.1
```

Both are dynamically linked against `libsqlite3.so.0` from `libsqlite3-0`; the one built against 9.1 also loads under 9.0.4.

5.5.1 With 3.53.4-2 or older

A second sqlite3 next to it only helps with the same number and at the front of `auto_path`; with different numbers `package require` takes the highest. So build the same version for both Tcl, from the sources of the Tcl distribution (for 9.1.0 `sqlite3.53.4.tar.gz`, a TEA package):

```
cd sqlite3.53.4
mkdir b86 b90
(cd b86 && ../configure --with-tcl=/usr/lib/tcl8.6 --libdir=$HOME/lib/tcltk \
        && make && make install-lib-binaries)
(cd b90 && ../configure --with-tcl=/usr/lib/tcl9.0 --libdir=$HOME/lib/tcltk \
        && make && make install-lib-binaries)
```

- `install-lib-binaries`, not `install-binaries`: the latter also wants to put `sqlite3_analyzer` into `/usr/bin`.

- Build against 9.0: the library then loads under 9.0 and 9.1.
- Both libraries belong in the same directory; the generated index picks by version.

`~/lib/tcltk` goes to the front via `TCLLIBPATH`. If a package with a higher number arrives later, the package wins again.

5.5.2 Own build and package

An own build and Debian's package do not have the same `PRAGMA compile_options`. Two of them change behavior:

Option	own build 3.53.4	Debian 3.54.1-1
DQS=0	set: SELECT "abc" is an error	missing: returns abc
USE_URI	missing: file:... without -uri 1 becomes a file name	set: URI

The program fixes the behavior itself, the same in every build:

```
sqlite3 db $datei -uri 1      ;# file: names always as URI
db config dqs_dml 0           ;# "text" in SELECT/INSERT … is an error
db config dqs_ddl 0           ;# likewise in CREATE TABLE …
```

`tdbc::sqlite3` is pure Tcl and needs no build of its own. Under 8.6 Debian's module loads, under Tcl 9 any `sqlite3-1.1.15.tm` in a tm path of the version, for example via `TCL9_0_TM_PATH`. If the program has loaded a package from an index before, the index wins at the same version (section *Which copy wins*).

5.6 TCLLIBPATH

TCLLIBPATH is put at the **front** of `auto_path` at startup. The value is a Tcl list, separated by spaces, not by colons like PATH. Container check, both versions the same:

TCLLIBPATH	Start of auto_path
/x/eins /x/zwei	/x/eins /x/zwei ...
/x/eins:/x/zwei	/x/eins:/x/zwei ..., a single entry

The second entry is a path that does not exist. Tcl reports nothing, the package stays unfindable.

TCLLIBPATH applies to **every** Tcl that the session starts. Anyone who adds directories with Tcl 9 packages to it also gives them to `tclsh8.6`. Tcl searches one level below each entry for `pkgIndex.tcl`; a `.so` for Tcl 9 fails under 8.6 at `load`. Cleaner is to keep a separate directory per version and extend `auto_path` in the program:

```
set hier [file dirname [file normalize [info script]]]
set auto_path [linsert $auto_path 0 [file join $hier lib]]
```

5.7 tm paths

Modules are not searched in `auto_path`, but in the tm path (`tcl::tm::path list`). Container check, Tcl 9.0.4 from `/opt/tcl9`:

```
/opt/tcl9/lib/tcl9/site-tcl
/opt/tcl9/lib/tcl9/9.0
//zipfs:/lib/tcl/tcl9/site-tcl
//zipfs:/lib/tcl/tcl9/9.0
```

The Ubuntu `tclsh8.6` reports 25 entries: `site-tcl` and the directories `8.0` to `8.6` each under `/usr/lib/tcltk/tcl8`, `/usr/lib/tcl8` and `/usr/share/tcltk/tcl8`, plus `/usr/share/tcltk/tcl8.6/tcl8`.

Additional locations come from environment variables whose name carries the version: TCL9_0_TM_PATH for Tcl 9.0, `TCL8_6_TM_PATH` for 8.6, plus `TCL8_5_TM_PATH` and so on for older minor versions, which 8.6 also reads. Tcl 9.1 reads `TCL9_1_TM_PATH` and also `TCL9_0_TM_PATH`. For each name there is a spelling with a dot (`TCL9.0_TM_PATH`), which `tm.tcl` also reads; in `bash` it cannot be exported. These variables separate with a **colon**, unlike `TCLLIBPATH`. Container check:

- `TCL9_0_TM_PATH=/x/tm90a:/x/tm90b`: Tcl 9.0.4 puts `/x/tm90b` and `/x/tm90a` at the top, in this reversed order.
- `TCL9_TM_PATH` and `TCL8_TM_PATH` (without minor version) are read by neither version.
- `TCL8_6_TM_PATH` is not seen by Tcl 9.0.
- Ubuntu's 8.6.14 does not put the entries from `TCL8_6_TM_PATH` at the top, but behind the eight directories under `/usr/lib/tcltk/tcl8` (checked with `TCL8_5_TM_PATH` and `TCL8_6_TM_PATH`: index 8 to 10 of 28). Modules of the same name and same version in these directories

therefore win; a higher version wins regardless of the directory.

The reversed order follows from the fact that `tcl::tm::path add` inserts each entry at the front. Anyone who needs a particular order sets it in the program with `tcl::tm::path add`.

5.8 Which copy wins

If a package exists as a module in the tm path and at the same time behind a `pkgIndex.tcl` in `auto_path`, the copy that is loaded depends on what the program requested before:

- If no index has been read yet, a matching module ends the search.
- The first search for a package that is not a module reads every `pkgIndex.tcl` in all directories of `auto_path`. After that, Tcl knows all versions from them. If one of them matches, a later call no longer asks for modules.
- With the same version, the index then wins. If the previously searched package lay in a different namespace directory than the module, the index wins even with a lower version: the module handler only searched in that directory, `struct/` for `struct::list`, the root for `tablelist`.

Checked with 8.6.14, 9.0.4 and 9.1.0, with the same result. The full table and a script to reproduce it are in *Ausliefern*, chapter Suchpfade.

On Linux this mainly affects old copies in directories from `TCLLIBPATH`. A typical case: `~/lib/tcltk` still holds an earlier installation with `pkgIndex.tcl`, the new one is in the same version as a module in `site-tcl`. A program that first loads `tablelist` (behind an index) gets the old copy afterwards; a program that only requests the module gets the new one. The remedy is to remove the old copy.

5.9 Which package was loaded?

```
package require struct::list
puts [package ifneeded struct::list [package present struct::list]]
```

The line shows the command Tcl would use to load the package, and thus its path. That is the quickest answer to "why does my Tcl 9 load the old package": usually a directory from TCLLIBPATH comes first.

For packages with a C part, info loaded is the proof. It names every library the interpreter has actually loaded:

```
package require sqlite3
puts [info loaded]
```

The report from `tclutils::tuappinfo` shows everything at once: the search variables (TCLLIBPATH, TCL*_TM_PATH, LD_LIBRARY_PATH) entry by entry, `auto_path`, `tcl_pkgPath` and the tm path with a mark `(missing)` on every directory that does not exist, every loaded package with its file and the list from `info loaded`. The example of the library loads the named packages first:

```
tclsh9.0 examples/demo-tuappinfo.tcl sqlite3
```

5.10 Summary

- Debian Tcl: `auto_path` via `/usr/share/tcltk` and `/usr/lib/tcltk/<multiarch>`, own packages into `/usr/local/…/tcltk`.
- Debian's Tcl 9 sees the same package directories as 8.6. Whether a package loads is decided by its index.
- `libsqlite3-tcl` up to 3.53.4-2 is built only for 8.6, from 3.54.1-1 also for Tcl 9. For older ones: build the same version for 8.6 and 9.0 and put it at the front of `auto_path`.
- Tcl 9 under `/opt` sees no distribution packages.
- TCLLIBPATH: Tcl list with spaces, applies to every Tcl of the session.
- TCL9_0_TM_PATH, TCL8_6_TM_PATH: colon, per version, inserted in reverse; 9.1 also reads TCL9_0_TM_PATH.
- A matching `.tm` ends the search as long as no index has been read. After that, with the same version, the index wins; after a package from a different namespace directory, even an older one does.
- Origin of a package: `package ifneeded` with `package present`; loaded libraries: `info loaded`.

Next chapter: Encoding and locale

6 Encoding and locale

6.1 Introduction

On Linux, the locale determines the encoding, not the operating system. At startup Tcl reads LC_ALL, LC_CTYPE and LANG and sets encoding system from them. If the locale is C or missing, that is not UTF-8. Three things depend on this: how `open` reads a file, how `puts` writes to the terminal, and under Tcl 8.6 also how `source` reads a script.

The long version of the topic, with Windows and TIP 656, is in the book *Encoding*.

6.2 encoding system follows the locale

`serie-linux` (howto 08) shows on Debian forky with Tcl 8.6.18:

```
LANG=C         encoding system = iso8859-1
LANG=C.UTF-8   encoding system = utf-8
```

The container check confirms this and adds further values. Each run with `env -i`, that is, without any other variable. Evaluated: `encoding system`, the length of a UTF-8 literal `"Grüße"` in the script, and the length of the same five characters, read from a UTF-8 file with `open` without `fconfigure`.

Environment	8.6.14 system	8.6.14 literal	9.0.4 system	9.0.4 literal	open, both
no locale	iso8859-1	7	iso8859-1	5	7
LANG=C	iso8859-1	7	iso8859-1	5	7
LANG=POSIX	iso8859-1	7	iso8859-1	5	7
LANG=C.UTF-8	utf-8	5	utf-8	5	5
LANG=de_DE.UTF-8	utf-8	5	utf-8	5	5
LANG=en_US.ISO-8859-1	iso8859-1	7	iso8859-1	5	7
LANG=de_DE	iso8859-1	7	utf-8	5	7 and 5 respectively
LANG=C.UTF-8 LC_ALL=C	iso8859-1	7	iso8859-1	5	7
LANG=C LC_CTYPE=C.UTF-8	utf-8	5	utf-8	5	5

What follows from this:

- The order is `LC_ALL`, then `LC_CTYPE`, then `LANG`. An `LC_ALL=C` beats any `LANG`.
- The locale does not have to be installed. The container has only `C`, `C.utf8` and `POSIX` (`locale -a`), yet `LANG=de_DE.UTF-8` leads to `utf-8`. Tcl reads the name.
- A locale without a character set (`de_DE`) is handled differently by 8.6.14 and 9.0.4: 8.6 takes iso8859-1, 9.0 takes utf-8.
- `open` follows `encoding system` in both versions.
- Tcl 9 always reads scripts as UTF-8. Tcl 8.6 reads them in `encoding system`.

6.3 Files: always with an encoding

Because the default depends on the locale, a program opens every text file with an explicit encoding. This applies under 8.6 and 9:

```
set ch [open $pfad r]
fconfigure $ch -encoding utf-8
set text [read $ch]
close $ch
```

`open` itself has no option for the encoding, not even under Tcl 9. The `fconfigure` line stays.

Tcl 9 aborts when reading invalid bytes instead of silently reinterpreting them (TIP 656, profile `strict` as the default for channels). Container check with a Latin-1 file `Gr\xfc\xdfe`, read with `-encoding utf-8`:

```
8.6.14   Grüße
9.0.4    error reading "file5": invalid or incomplete multibyte or wide character
```

Tcl 8.6 interprets invalid UTF-8 bytes as Latin-1. Here that happens to look right and hides the fact that the file is not UTF-8 at all. Under Tcl 9 the same error becomes visible. On Linux this mainly affects old files and output of other programs. The profiles (`-profile replace`, `-profile tcl8`) are described in the book *Encoding*.

6.4 Scripts: the guard under Tcl 8.6

A UTF-8 script with umlauts in the source only runs correctly under Tcl 8.6 if the locale is UTF-8. Under `LANG=C`, `Rückgängig` becomes the string `RÃ¼ckgÃ¤ngig` (`serie-linux`, howto 08). This affects programs that start from cron, from a systemd unit or from a container without a locale.

A remedy without startup parameters is a guard at the start of the file. The file itself is UTF-8. If Tcl reads it as iso8859-1, `"ü"` is two characters long. Then the guard reads the file again, this time as UTF-8:

```
if {[string length "ü"] != 1} {
    source -encoding utf-8 [file normalize [info script]]
    return
}
```

`beispiele/utf8-wache.tcl` shows this. Container check, under `LANG=C` and `LANG=C.UTF-8`, with 8.6.14 and 9.0.4: in all four runs `"Rückgängig"` has length 10. Under `LANG=C` the terminal still shows the umlauts wrongly, because `stdout` writes in iso8859-1. The string in the program is correct.

The guard belongs in the startup script. For further files there is `source -encoding utf-8` directly, for packages the entry in `pkgIndex.tcl`:

```
package ifneeded meinpaket 1.0 \
    [list source -encoding utf-8 [file join $dir meinpaket.tcl]]
```

Alternatively, the start command passes the encoding explicitly: `tclsh8.6 -encoding utf-8 app.tcl`. That only works if you control the call yourself, that is, not via a shebang line with `env` (see Which Tcl?).

6.5 Terminal and exec

`stdout`, `stderr` and `stdin` are set to `encoding system` at startup (container: `fconfigure stdout -encoding` returns `utf-8` under `C.UTF-8`). The same applies to

channels to child processes: `exec` and `open "|…"` decode the child's output in `encoding system`. `exec printf %s "Grüße"` returns `Grüße` under `LANG=C.UTF-8`. A child that always writes UTF-8 is read, under an unknown locale, via a channel with a fixed encoding:

```
set ch [open |[list git log -1 --format=%an] r]
fconfigure $ch -encoding utf-8
set autor [string trim [read $ch]]
close $ch
```

See exec, xdg-open, processes.

6.6 msgcat and language

`msgcat::mclocale` also reads the locale variables, in the order `LC_ALL`, `LC_MESSAGES`, `LANG` (source of `msgcat.tcl` in Tcl 9.0.4). In the container with `LANG=C.UTF-8` the result is `c`. A program with German messages in `de.msg` therefore needs a German locale or an explicit call `msgcat::mclocale de`. `LANGUAGE`, on Linux the language list for gettext, is not evaluated by `msgcat`.

6.7 Summary

- `encoding system` follows `LC_ALL`, `LC_CTYPE`, `LANG`. `C` and a missing locale result in iso8859-1.
- `open` follows `encoding system`. Always open files with `fconfigure -encoding utf-8`.
- Tcl 9 reads scripts as UTF-8, Tcl 8.6 in `encoding system`. For 8.6, the guard at the start of the startup script.
- de_DE without a character set: 8.6 iso8859-1, 9.0 utf-8.
- Read output of child processes via a channel with a fixed encoding.

Next chapter: exec, xdg-open, processes

7 exec, xdg-open, processes

7.1 Introduction

On Linux, `exec` is the tool for everything Tcl cannot do itself: open files with the matching program, start the browser, query system values. There is no shell in between. Tcl starts the program directly, with the arguments as a list. That makes `exec` safe against spaces and special characters, but shell syntax like `*`, `~` or `$VAR` has no effect.

7.2 Finding a program: auto_execok

auto_execok searches for a program in `PATH` and returns the full path as a list, or an empty string:

```
set p [auto_execok xdg-open]
if {$p eq ""} {
    error "xdg-open fehlt"
}
exec {*}$p $datei &
```

In the container: `auto_execok sh` returns `/usr/bin/sh`, `auto_execok xdg-open` returns `/usr/bin/xdg-open`, `auto_execok lpstat` an empty string (no CUPS). The result is a list, hence the `{*}`. On Windows it can have several elements, on Linux it is one path.

`auto_execok` remembers every answer in the global array `auto_execs`, including the empty one. If you install a tool after the program has started, delete the entry, otherwise the old answer remains:

```
unset -nocomplain ::auto_execs(xdg-open)
```

`auto_reset` also empties the array, but at the same time removes all commands loaded via `auto_index` (source `auto.tcl` in Tcl 9.0.4). For this purpose it is too coarse.

7.3 Telling exec errors apart

`exec` throws an error in three different cases. Which one applies is in `errorCode`. Container check, 8.6.14 and 9.0.4 the same:

Case	Message	errorCode
Program missing	couldn't execute "gibtsnicht": no such file or directory	POSIX ENOENT {no such file or directory}
Exit status not 0 (false)	child process exited abnormally	CHILDSTATUS <pid> 1
Output on stderr	the stderr text, here meldung	NONE
Child terminated by signal	child killed: software termination signal	CHILDKILLED <pid> SIGTERM {software termination signal}

The third case is the well-known pitfall: a program that runs successfully but writes a warning to stderr counts as an error for `exec`. Remedy:

```
exec -ignorestderr programm arg      ;# stderr goes to Tcl's stderr
exec programm arg 2>@1               ;# stderr into the result
exec programm arg 2>/dev/null        ;# discard stderr
```

A pattern for all cases:

```
try {
    set aus [exec -ignorestderr {*}$befehl]
} trap {CHILDSTATUS} {msg opts} {
    set rc [lindex [dict get $opts -errorcode] 2]
    puts "Rückgabewert $rc"
} trap {POSIX ENOENT} {} {
    puts "Programm fehlt"
}
```

7.4 In the background and via channels

An `&` at the end starts the program in the background. `exec` does not wait and returns the process IDs:

```
set pids [exec xdg-open $datei &]
```

For long-running programs whose output you want to read without halting the user interface, a channel is the right tool:

```
set ch [open |[list ping -c 3 localhost] r]
fconfigure $ch -blocking 0 -encoding utf-8
fileevent $ch readable [list lesen $ch]
proc lesen {ch} {
    if {[gets $ch zeile] >= 0} { .t insert end $zeile\n }
    if {[eof $ch]} {
        fconfigure $ch -blocking 1
        catch {close $ch} fehler
    }
}
```

The `fconfigure -blocking 1` before the `close` makes sure that `close` reports the child's exit status. Otherwise the channel's encoding follows `encoding system`, see Encoding and locale.

7.4.1 The pitfall with your own selection

A blocking `exec` halts the event loop. Usually that is merely unpleasant. It becomes a deadlock when the child waits for your own program. Container check: a `wish` selects text in an entry and then calls `exec xclip -o -selection primary`. `xclip` asks the owner of the selection, that is, this `wish`. It is stuck in `exec` and does not answer. Both wait until `timeout` ends the run after 20 seconds. With a channel and `fileevent` as above, the same call goes through (`beispiele/auswahl-probe.tcl`). More on the selection in Mouse, keyboard, clipboard.

7.5 xdg-open: opening a file or address

xdg-open is the Linux counterpart to `start` on Windows and `open` on macOS. It opens a file with the configured program or an address in the browser (`serie-linux`, howto 07):

```
proc oeffnen {ziel} {
    set p [auto_execok xdg-open]
    if {$p eq ""} { error "xdg-open fehlt (Paket xdg-utils)" }
    exec {*}$p $ziel &
}
oeffnen https://www.tcl-lang.org
oeffnen [file join $env(HOME) bericht.pdf]
```

`xdg-open` belongs to the package `xdg-utils`. It asks the desktop environment which program is responsible. Under Xfce it passes on to `exo-open` (see Xfce from Tcl). The association for a file type is shown by `xdg-mime query default <MIME-Typ>`.

The `&` is important. Whether `xdg-open` returns immediately or waits until the opened program ends depends on the desktop environment and on the program; without `&` the user interface can freeze. Checked with Tcl 8.6:

Call	Workstation, Xfce	Container without desktop
exec xdg-open /etc/hostname &	process ID immediately, Mousepad opens	calls LibreOffice and waits

Call	Workstation, Xfce	Container without desktop
exec xdg-open https://example.org &	process ID immediately, browser opens	no browser, exit 3
exec xdg-open /gibtsnicht	CHILDSTATUS … 4	CHILDSTATUS … 2

The exit codes are listed in `xdg-open(1)`: 2 file missing, 3 no program, 4 action failed. Under Xfce, `xdg-open` does not check the file itself, but passes it on to `exo-open`; its error becomes 4. Without a desktop session (`xdg-utils` 1.1.3, container) it checks itself and reports 2. Code should therefore only check for "not 0", not for a specific number. The windows do not appear in the order of the calls: on the workstation the browser came before Mousepad, although `/etc/hostname` was opened first. If you expect a window afterwards, do not wait for a particular order. The series do not start `xdg-open` under `SERIES_AUTO`.

7.6 Process ID and signals

```
pid                     ;# own process ID
pid $ch                 ;# process IDs behind a channel
```

Tcl does not send a signal itself. `exec kill -TERM $p` does. A Tcl program that is to react to `SIGTERM` needs an extension (such as `TclX` with `signal`); without one it ends immediately. This has not been checked.

On Linux every program has `/proc/<pid>/` with `cmdline`, `environ` and `status`. Whether a process is still alive:

```
file isdirectory /proc/$p
```

7.7 Summary

- `auto_execok` before every `exec` of an external program, insert the result with `{*}`.
- `errorCode` distinguishes: `POSIX ENOENT`, `CHILDSTATUS`, `NONE` (stderr), `CHILDKILLED`.
- `-ignorestderr` for programs that chatter on stderr.
- Long runs via `open |…` with `fileevent`. Never call a program blocking that asks your own program.
- `xdg-open` with `&`, package `xdg-utils`.

Next chapter: X11, window managers and Wayland

8 X11, window managers and Wayland

8.1 Introduction

On Linux, Tk speaks only X11. `tk windowingsystem` always returns `x11`. Unlike on Windows and macOS, the windowing system is split into two programs: the X server, which draws and delivers input, and the window manager, which provides the frame, title bar, focus and iconify. A Tk program can run without a window manager, but then it lacks a few things. This chapter separates what Tk does itself, what the window manager does, and what a Wayland session changes about it.

8.2 DISPLAY and starting without a display

At startup, Tk connects to the X server given in DISPLAY. If the variable is missing or the server does not answer, Tk fails. Container check, 8.6.14 and 9.0.4 identical:

```
$ env -i wish9.0 probe.tcl
application-specific initialization failed: no display name and no $DISPLAY
    environment variable
$ env -i DISPLAY=:77 wish9.0 probe.tcl
application-specific initialization failed: couldn't connect to display ":77"
```

What happens next is remarkable: `wish` does **not** abort. It runs the script without Tk, `package require Tk` then returns the error, and the exit status of `wish` is 0. A test run in a pipeline that only looks at the exit status reports success, although not a single line of the user interface has run. Remedy at the start of the script:

```
if {[catch {package require Tk} fehler]} {
    puts stderr "Tk fehlt: $fehler"
    exit 1
}
```

`tclsh` with `package require Tk` behaves as expected: the error aborts the script.

What Tk knows about the server:

```
winfo server .          ;# X11R0 The X.Org Foundation 12101011 (Xvfb)
winfo screen .          ;# :93.0
winfo screenwidth .     ;# 1280
winfo screenmmwidth .   ;# 325
winfo visual .          ;# truecolor
```

8.3 Window class and WM_CLASS

Every main window carries the X property WM_CLASS with two parts: name and class. The window manager, the taskbar and `.desktop` files recognize a program by it. Tk derives both from the program name. Container check with `xprop`, 8.6.14 and 9.0.4 identical:

Start	tk appname	WM_CLASS
wish9.0 meinprog.tcl	meinprog.tcl	"meinprog.tcl", "Meinprog.tcl"
wish9.0 meinprog.tcl -name meinname	meinname	"meinname", "Meinname"
tclsh9.0 meinprog with package require Tk	meinprog	"meinprog", "Meinprog"

The class is the name with a capital first letter. Tk does not accept a custom class name for. from the command line: a `-class` in `argv` is ignored and stays in `argv`. `-name`, on the other hand, also works when a script sets it in `::argv` before `package require Tk`:

```
set ::argv [list -name meineapp]
set ::argc 2
package require Tk          ;# WM_CLASS "meineapp", "Meineapp"
```

For a program started as `meineapp.tcl`, the class is therefore `Meineapp.tcl`, with the

extension. Anyone writing a `.desktop` file with `StartupWMClass` must match exactly this value, see Native dialogs and desktop integration. Further toplevels carry their window name and their class: a `toplevel .t -class Zweit` has `WM_CLASS "t", "Zweit"` in the container (8.6.14 and 9.0.4). For `.` itself there is no such way.

Tk also sets `WM_NAME` and `_NET_WM_NAME` from `wm title`, and `_NET_WM_PID` as soon as `WM_CLIENT_MACHINE` is set (source `tkUnixWm.c`). In the container, `_NET_WM_PID` was missing on the window; the condition for it has not been investigated.

8.4 What the window manager provides

A reparenting window manager puts every main window into a frame of its own with a title bar. This results in two sets of coordinates: `wm geometry` sets and returns the **frame corner**, `winfo rootx` and `winfo rooty` the corner of the **client area**. The difference is the frame.

```
wm geometry . 360x220+30+40
update
scan [wm geometry .] {%dx%d+%d+%d} w h gx gy
set titel [expr {[winfo rooty .] - $gy}]
```

Measured in `serie-xvfb` (howto 04 and 13, Tcl/Tk 8.6.14 and 9.0.4):

Environment	Title bar rooty - gy	wm frame . vs. winfo id .
Xvfb without window manager	0	equal
Xvfb with openbox	20 (sides 1)	different
Xfce on the author's workstation	29 (sides 5)	different

Under Xfce the same for 8.6.18, 9.0.4 and 9.1.0; example 9.0.4: `wm frame 0x92e055`, `winfo id 0x7a00009`.

A fixed number such as 80 for "below the main window" fits none of these environments. To place a dialog below the title bar of the main window, use `winfo rooty .`.

What is missing without a window manager, container check with `beispiele/wm-probe.tcl`, 8.6.14 and 9.0.4 identical:

Operation	without window manager
Focus after update	empty, nobody assigns it
wm iconify .	state stays normal, Tk waits about 2000 ms
wm attributes . -fullscreen 1	attribute stays 0, size unchanged
focus -force .	works
grab	works (Tk-internal, serie-xvfb, howto 11)

With openbox, the window becomes `iconic` after `wm iconify`, and `.` usually already has the focus after `update` (`serie-xvfb`, howto 09 and 10). Tk sets fullscreen, `-zoomed` and `-type` via the EWMH properties `_NET_WM_STATE` and `_NET_WM_WINDOW_TYPE`. Whether they take effect is up to the window manager. `wm attributes . -alpha 0.8` sets `_NET_WM_WINDOW_OPACITY`, namely on the wrapper window that Tk puts around the client

area (source `tkUnixWm.c`). Under a reparenting window manager, this is neither `wm frame` nor `winfo id`: with openbox, the property is on the wrapper window and on the frame that openbox supplies with it, not on the client (container, 8.6.14 and 9.0.4). `xwininfo` names the wrapper window as the parent window of the client:

```
regexp {Parent window id: (0x[0-9a-f]+)} [exec xwininfo -tree -id [winfo id .]] ->
    huelle
exec xprop -id $huelle _NET_WM_WINDOW_OPACITY
```

Under Xfce 4.20 with compositor (`use_compositing` true), `-alpha` accepts 0.35 and returns 0.35, and the window is visibly transparent; with `use_compositing` false it is opaque (8.6.18, 9.0.4, 9.1.0). The property is not on the frame or on the client, but it is on the wrapper window: 1503238553, i.e. 0.35 × 0xFFFFFFFF, for 8.6.18, 9.0.4 and 9.1.0, with and without compositor. So Tk always sets it; whether anything is visible is decided by the compositor alone.

The attributes known under X11, container check:

```
8.6.14   -alpha 1.0 -topmost 0 -zoomed 0 -fullscreen 0 -type {}
9.0.4    -alpha 1.0 -fullscreen 0 -topmost 0 -type {} -zoomed 0
```

Same content, different order. When comparing the output as a list, compare it as a `dict`.

8.5 Detecting a window manager

`winfo manager.` does not help: for a toplevel it is always `wm`, with and without a window manager (`serie-xvfb`, howto 02). What works is shown by `serie-xvfb` in howto 09:

1. `wm frame.` vs. `winfo id.`. Different means: a reparenting window manager is running.
2. If they are equal, an iconify probe on a helper window decides. If it becomes `iconic`, a window manager without frames is running (dwm, xmonad). If it stays `normal`, none is running.

Compare the ids **numerically**. Tk 8.6 writes `winfo id` with leading zeros, `wm frame` without (container: `0x200003` vs. `0x00200003`), Tk 9.0 writes both without. Before the first mapping, `wm frame` is `0x0`, so call `update` first.

```
update
set rahmen [expr {[wm frame .] != [winfo id .]}]
```

Without a window manager, the iconify probe costs about two seconds. The procedures `serie::wmRahmen` and `serie::wmDa` in the series therefore allow the result to be preset via `SERIE_WM=0` or `1`.

8.6 Wayland

Tk 9.0.4 has no Wayland backend. The source mentions Wayland in exactly two places: in `scaling.tcl` (see Fonts, scaling, themes) and in `tkUnixEvent.c`, where Tk resets a nonsensical screen width in millimeters under XWayland to 75 dpi.

In a Wayland session, a Tk program therefore runs via **XWayland**, the X server that the Wayland compositor brings along. Then `DISPLAY` and `WAYLAND_DISPLAY` are both set, and `XDG_SESSION_TYPE` is `wayland`. `tk windowingsystem` stays `x11`.

```
set sitzung [expr {[info exists env(XDG_SESSION_TYPE)]
                   ? $env(XDG_SESSION_TYPE) : "unbekannt"}]
```

What is different under XWayland (global window positions, taking focus from other programs, screenshots, scaling with multiple screens) depends on the compositor. On the author's workstation, Xfce runs under X11 (`serie-xfce`: `XDG_SESSION_TYPE=x11`, `WAYLAND_DISPLAY` empty); the Debian book also reports `x11` for the Debian workstation. A real Wayland session (GNOME, KDE, Xfce with Wayland) has **not been checked** for this book.

8.6.1 XWayland under Weston

XWayland has been checked in the container: Weston 13.0.0 nested in Xvfb (`weston --backend=x11-backend.so --xwayland`), Tk via `DISPLAY=:0`, 8.6.14 and 9.0.4 identical:

Probe	under XWayland (Weston)
tk windowingsystem	x11
wm frame . vs. winfo id .	different, Weston sets a frame
wm geometry . +100+100	Weston places the window itself, wm geometry reports +32+32
wm geometry . +400+300	reports +400+300, the window does not move
winfo rootx/rooty	6,27 for every window, fixed
focus -force .	no effect, the focus stays with the window shown last
wm iconify .	state stays normal
-fullscreen 1	works, whole output
xwd -root	BadMatch, no image

Under XWayland, window positions can therefore be neither set nor read: Tk reports what it requested, not where the window is. Code that places dialogs itself or takes screenshots via X should not rely on this. Other compositors may behave differently.

8.7 Summary

- `tk windowingsystem` is always `x11` on Linux, even under Wayland (via XWayland). There, positions can be neither set nor read, no iconify, no `focus -force` (Weston).
- Without a display, `wish` reports the error, continues without Tk and exits with 0. Scripts check `package require Tk` themselves.
- `WM_CLASS` is the program name and the program name with a capital first letter; `-name` changes both.
- `wm geometry` is the frame corner, `winfo rooty` the client area. Frame: 0, 20 or 29 pixels, never assume a fixed value.
- Without a window manager: no focus, no iconify, no fullscreen. Detect via `wm frame` and an iconify probe.

Next chapter: Fonts, scaling, themes

9 Fonts, scaling, themes

9.1 Introduction

Under X11, Tk draws text with Xft and looks up fonts via fontconfig. The size of the user interface depends on `tk scaling`, and that depends on the resolution the X server reports. Colors and themes come from Tk itself and from the X resource database, not from the desktop settings. This chapter shows where each value comes from and what Tk 9 has changed about scaling.

9.2 Fonts: Xft and fontconfig

Whether Tk is built with Xft is reported by `tk::pkgconfig get fontsystem`. In the container it returns `xft` under 8.6.14 and 9.0.4. Tk then sets the named fonts to the fontconfig names `sans-serif` and `monospace` (source `ttk/fonts.tcl`). Which family is behind them is decided by fontconfig on the machine the program runs on:

```
fc-match sans-serif     # Container: Inter-Regular.otf: "Inter" "Regular"
fc-match monospace      # Container: DejaVuSansMono.ttf: "DejaVu Sans Mono" "Book"
```

Container check, both versions identical:

Name	Family	Size
TkDefaultFont, TkTextFont, TkMenuFont, TkIconFont	Inter	10
TkHeadingFont	Inter, bold	10
TkCaptionFont	Inter, bold	12
TkTooltipFont, TkSmallCaptionFont	Inter	9
TkFixedFont	DejaVu Sans Mono	10

`serie-xvfb` (howto 01) also reports `TkDefaultFont Inter 10`. On a machine without Inter, fontconfig returns a different family, usually DejaVu Sans. The text must not assume it.

The sizes are **points**, not pixels. A requested name that does not exist does not lead to an error: `font actual {GibtsNicht 10}` returns `Inter` in the container. `Helvetica` becomes `TeX Gyre Heros`, because fontconfig knows a substitute for it. To check whether a family is really there, ask `font families`:

```
if {"Noto Sans" in [font families]} {
    font configure TkDefaultFont -family "Noto Sans"
}
```

The fonts come from the **client**, not from the X server. A program that runs on another machine via `ssh -X` draws with the fonts of the remote machine. The same applies to Xvfb: the virtual server has no fonts of its own in the Xft sense.

9.3 tk scaling and DPI

tk scaling is the number of pixels per point. Tk computes it at startup from the screen size in pixels and millimeters: pixels ÷ (mm ÷ 25.4) ÷ 72. At 100 dpi that is 1.389. All sizes in points,

including the fonts, grow with this value.

`serie-xvfb` (howto 01, Tcl/Tk 8.6.14 and 9.0.4) measures this with different `-dpi` values of Xvfb:

Xvfb	tk scaling
1280x1024, without -dpi	1.389 (≈ 100 dpi)
-dpi 50	0.695
-dpi 200	2.770

Container check with a label "Hallo Welt" in `TkDefaultFont` and a `ttk::scrollbar`:

Xvfb	Version	tk scaling	::tk::scalingPct	Line height	Text width	Scrollbar
100 dpi	8.6.14	1.389	missing	18	69	15
100 dpi	9.0.4	1.389	100	18	69	15
192 dpi	8.6.14	2.667	missing	33	131	15
192 dpi	9.0.4	2.667	200	33	131	27

The font grows in both versions. The scrollbar grows only under Tk 9: Tk 9 computes many dimensions in ttk and in the standard dialogs via ::tk::scalingPct, the resolution as a percentage of 96 dpi, rounded to 25 and at least 100. Under 8.6, pixel dimensions such as scrollbar width and checkbox stay fixed.

9.3.1 Where Tk 9 gets the scaling from

A desktop usually does not set the magnification via the DPI of the X server, but via its own settings. Tk 9 reads some of them at startup (source `library/scaling.tcl`, 9.0.4), in this order:

1. If `xfce4-session` is running: `xfconf-query -c xsettings -p /Gdk/WindowScalingFactor`. From factor 2 on, 200 % applies, and the font sizes are doubled (`::tk::fontScalingFactor` 2).
2. If `mate-session` is running: `gsettings` for MATE.
3. If `gnome-session` is running: the key `Gdk/WindowScalingFactor` in the xsettings overrides.
4. Otherwise the X resource Xft.dpi, read with `xrdb -query`: percent = `Xft.dpi` ÷ 96 × 100.
5. Under GNOME without Wayland, finally `~/.config/monitors.xml`.

If the value found differs from the measured one, Tk 9 sets `tk scaling` anew. Container check: Xvfb with 100 dpi, plus `Xft.dpi: 192` via `xrdb -merge`:

Version	tk scaling	scalingPct	font actual … -size	Line height	Scrollbar
8.6.14	1.389	missing	19	33	15
9.0.4	2.672	200	10	33	27

Under 8.6, Xft enlarges the font itself, because fontconfig knows `Xft.dpi`. Tk knows nothing about it: `tk scaling` stays 1.389, and `font actual` reports the back-calculated size 19 instead of 10. Pixel dimensions stay small. Under 9.0, font, computed value and dimensions agree.

To set the scaling yourself, set it before building the user interface:

```
tk scaling 2.0     ;# 144 dpi; before the first widget
```

Under Tk 9, `tk scaling` immediately updates `::tk::scalingPct` as well: `tk scaling 2.0` gives 150 (factor × 75, rounded to 25; source `tkCmds.c`). How much a standard dialog grows depends on the font. `tk_messageBox` before and after `tk scaling 2.0`:

Environment	Version	scalingPct	Dialog	Line height TkDefaultFont
Container, Xvfb 100 dpi	8.6.14	missing	117×95 → 162×114	18 → 25
Container, Xvfb 100 dpi	9.0.4	100 → 150	117×95 → 164×116	18 → 25
Workstation, Xfce	8.6.18	missing	110×96 → 110×96	34 → 34
Workstation, Xfce	9.0.4	100 → 150	110×96 → 110×100	34 → 34
Workstation, Xfce	9.1.0	100 → 150	112×96 → 128×119	31 → 31

In the container the font grows, and the dialog with it. On the workstation the font stays the same; what grows there is computed by Tk via `scalingPct`: barely under 9.0, clearly under 9.1.

9.4 Themes

Under X11, Tk ships four ttk themes. Container check, both versions: `ttk::style theme names` returns `clam alt default classic`, the active one is `default`. There is no native Linux theme like `aqua` or `vista`. Tk does not adopt the look of GTK or Qt programs. There are add-on packages for that (such as `ttk::theme::*` collections); they are not part of this book.

The background of classic widgets is `#d9d9d9`. `TButton -padding` reports `3 3` under 8.6 and `2.25p` under 9.0: Tk 9 computes spacing in points so that it grows with `tk scaling`.

9.5 X resources

Under X11, Tk reads the resource database of the X server at startup, the property RESOURCE_MANAGER on the root window, which `xrdb` sets. If it is missing, Tk reads `~/.Xdefaults`. Container check, 8.6.14 and 9.0.4 identical:

```
*background:  lightyellow     -> option get . background Background: lightyellow
                                 button -background: lightyellow
*Label.font:  {DejaVu Sans} 14 -> label -font: {DejaVu Sans} 14
*TkTheme:     clam             -> ttk::style theme use: clam
```

So `*TkTheme` selects the ttk theme without a single line of code. The file `~/.Xdefaults` with `*background: pink` took effect in the container as long as no resources were loaded. With XENVIRONMENT set to a file, nothing changed.

This has two sides. A user can adjust color, font and theme of a Tk program this way. A program that looks strange on someone else's machine may find a `~/.Xdefaults` from old times there. `xrdb -query` shows what is loaded.

A program loads its own defaults with low priority so that the user can override them:

```
option add *Entry.background white widgetDefault
option readfile [file join $hier app.Xdefaults] startupFile
```

A peculiarity of Xvfb: without `-noreset`, the server resets its state as soon as the last client closes the connection. Resources that an `xrdb` loaded beforehand are then gone before the Tk program starts. In the container, `xrdb -query` stayed empty for this reason until Xvfb ran with `-noreset`. For tests with resources, `-noreset` belongs in the Xvfb options, see GUI tests with Xvfb.

9.6 Summary

- Xft and fontconfig: `sans-serif` and `monospace`, family from the machine the program runs on. Sizes in points.
- `tk scaling` from pixels and millimeters. Tk 9 additionally reads Xfce, MATE and GNOME settings and `Xft.dpi`, and also scales pixel dimensions.
- Under 8.6, `Xft.dpi` only enlarges the font.
- Themes: `clam alt default classic`, default `default`.
- X resources take effect: `*background`, `*TkTheme`. If `RESOURCE_MANAGER` is missing, `~/.Xdefaults` applies.

Next chapter: Mouse, keyboard, clipboard

10 Mouse, keyboard, clipboard

10.1 Introduction

Three places where Linux code looks different from code for Windows or macOS: the mouse wheel, which arrives as a mouse button under X11; the modifiers, whose names do not mean a fixed key under X11; and the two clipboards PRIMARY and CLIPBOARD. For the mouse wheel and for `Command`, Tk 8.6 and Tk 9 have made different decisions.

All values in this chapter were determined in the container with `beispiele/rad-und-tasten.tcl` and `beispiele/auswahl-probe.tcl`. The scripts use `xdotool` to send real X events to their own window. The keyboard layout of the Xvfb is the default `pc105`, `us`.

10.2 The mouse wheel

X11 has no wheel event. The X server reports each notch as a press and release of a mouse button: 4 up, 5 down, 6 and 7 for tilting sideways. Tk 8.6 passes this on as is. Tk 9 translates it into MouseWheel as on Windows and macOS (source `tkEvent.c`: buttons 4 to 7 become `MouseWheelEvent` on press, with `%D` equal to ±120, 6 and 7 additionally get Shift; the release is discarded).

Container check:

X button	Tk 8.6.14	Tk 9.0.4
4 (up)	<Button-4>, %b=4	<MouseWheel>, %D=120
5 (down)	<Button-5>, %b=5	<MouseWheel>, %D=-120
6 (left)	<Button-4>, %s=1 (Shift)	<Shift-MouseWheel>, %D=120
7 (right)	<Button-5>, %s=1 (Shift)	<Shift-MouseWheel>, %D=-120
8 (side button "Back")	<ButtonPress>, %b=8	<Button-4>, %b=8
9 (side button "Forward")	<ButtonPress>, %b=9	<Button-5>, %b=9

The last two rows are the pitfall. Tk 9 has freed the numbers 4 and 5 and assigned them to buttons 8 and 9. Old code with

```
bind .c <Button-4> {.c yview scroll -1 units}
bind .c <Button-5> {.c yview scroll  1 units}
```

no longer scrolls with the wheel under Tk 9, but with the side buttons of a mouse. An `event generate .l <Button-4>` also gives `%b=8` under Tk 9. Tk 8.6, on the other hand, rejects `<Button-8>` ("bad button number "8"").

The class bindings of `Text` (`bind Text`) show the same difference:

```
8.6.14  <Shift-Button-5> <Shift-Button-4>
        <Button-5> <Button-4>
        <Shift-MouseWheel> <MouseWheel>
9.0.4   <Shift-Alt-MouseWheel> <Shift-MouseWheel>
        <Alt-MouseWheel> <MouseWheel>
```

Your own code for both versions:

```
proc rad {w delta} {
    $w yview scroll [expr {$delta > 0 ? -1 : 1}] units
}
bind .c <MouseWheel> {rad %W %D}
if {[tk windowingsystem] eq "x11"
        && ![package vsatisfies [package provide Tk] 9-]} {
    bind .c <Button-4> {rad %W 120}
    bind .c <Button-5> {rad %W -120}
}
```

Under X11, `%D` is always a multiple of 120. Finer values, as touchpads deliver them under Windows and macOS, do not arrive under X11. Whether a touchpad under Xfce produces smooth values has not been checked.

10.3 Modifiers and keys

X11 has eight modifier bits: Shift, Lock, Control and Mod1 to Mod5. Which key sets which bit is defined by the keyboard layout (`xmodmap -pm`). In the container: Mod1 carries Alt_L, Alt_R and Meta_L, Mod2 Num_Lock, Mod4 Super_L and Super_R, Mod5 ISO_Level3_Shift (AltGr).

Tk has names for these that are meant to bridge platforms. Container check, each binding set individually:

Binding	8.6.14 fires on	9.0.4 fires on
<Control-c>	Ctrl+C	Ctrl+C
<Alt-c>	Alt+C	Alt+C
<Mod1-c>	Alt+C	Alt+C
<Mod4-c>	Super+C	Super+C
<Meta-c>	nothing	nothing
<Command-c>	Alt+C	nothing
<Option-c>	nothing	Alt+C
<Super-c>	error at bind	error at bind

Under Tk 8.6, Command under X11 is the same as `Mod1`. Under Tk 9.0.4 it is the same as `Meta` (source `tkBind.c`), and `Option` is `Alt`. Tk determines which bit `Meta` is from the layout: the bit on which a key with the keysym `Meta_L` or `Meta_R` lies in the first level (source `tkUnixKey.c`). In the container, `Meta_L` is on keycode 205, and its first level is `NoSymbol`:

```
$ xmodmap -pke | grep 'keycode 205'
keycode 205 = NoSymbol Meta_L NoSymbol Meta_L
```

So Tk finds no Meta bit. Then `Meta`, and with it `Command`, fires on no key.

The Debian book writes in the chapter *Tcl unter Linux*: "A binding `<Command-c>` from a Mac program therefore reacts to Alt+C on Linux." This holds for Tk 8.6, checked with 8.6.14. Under Tk 9.0.4 it reacts to no key in the container. The same on the author's workstation (Xfce, German layout): `xdotool` sends Alt+C to a window with only `<Command-c>`. 8.6.18 fires and stores the binding as `<Mod1-Key-c>`, 9.0.4 and 9.1.0 stay silent and store `<Meta-Key-c>`. There, as in the container, `Meta_L` is on keycode 205 with `NoSymbol` in the first level. The rule of the Debian book remains correct: bind `Command` only under `aqua`, under `x11` bind `Control`.

```
set mod [expr {[tk windowingsystem] eq "aqua" ? "Command" : "Control"}]
event add <<Speichern>> <$mod-s>
bind . <<Speichern>> speichern
```

The Super key (Windows key) is called `Mod4` in `bind`. The desktop only intercepts the Super shortcuts it has assigned itself. On the workstation (Xfce), Super+C reaches Tk, as `<Mod4-c>` with the keysym `Super_L`; the session there assigns Super+E, Super+P and Super+R, among others. Which ones are assigned is shown by `xfconf-query -c xfce4-keyboard-shortcuts -l -v`.

Tk reports the key itself with its keysym (%K): `Control_L`, `Alt_L`, `Super_L`. When a modifier is pressed, `%s` is still 0; the bit only appears with the next key.

10.4 Two clipboards

X11 has several "selections". Two are important for programs:

- PRIMARY: the most recently selected text. It is set without a copy command and pasted with the middle mouse button.
- `CLIPBOARD`: the clipboard in the sense of Windows and macOS. Set with Ctrl+C or `<<Copy>>`,

pasted with Ctrl+V or `<<Paste>>`.

`clipboard` in Tk means CLIPBOARD, `selection` without an option means PRIMARY. Container check, 8.6.14 and 9.0.4 identical, text "Auswahl-Ü" selected in the entry:

```
PRIMARY von innen:    Auswahl-Ü
PRIMARY über xclip:   Auswahl-Ü
CLIPBOARD vorher:     leer (CLIPBOARD selection doesn't exist or form "STRING" not
    defined)
CLIPBOARD nachher:    Auswahl-Ü        (nach event generate .e <<Copy>>)
CLIPBOARD über xclip: Auswahl-Ü
TARGETS:              MULTIPLE TARGETS TIMESTAMP TK_APPLICATION TK_WINDOW
    UTF8_STRING STRING
```

Selecting alone sets PRIMARY, not CLIPBOARD. Tk offers both as `UTF8_STRING` and `STRING`, so umlauts arrive in other programs.

The most important difference from Windows: an X11 selection is not stored content, but a promise by the program that owns it. When the program ends, the content is gone. In the container, `xclip -o -selection clipboard` returned only `Error: target STRING not available` after `wish` had ended. A clipboard manager such as `xfce4-clipman` takes over the content on copy and keeps it, but only if it is running. On the workstation the package 1.6.6-1 is installed, but no process is running; after `wish` ends, `xclip` reports the same there as in the container. So installed does not mean started: `pgrep xfce4-clipman`.

`serie-linux` (howto 11) shows: PRIMARY only exists after an X11 selection; otherwise `selection get -selection PRIMARY` fails. Code therefore asks with `catch`:

```
if {[catch {selection get -selection PRIMARY} text]} {
    set text ""
}
```

Anyone reading another program's selection with a tool such as `xclip` must not do so with a blocking `exec` while their own program is the owner. That deadlocks both, see exec, xdg-open, processes.

10.4.1 Virtual events

Container check, `event info`:

Event	8.6.14	9.0.4 additionally
<<Copy>>	<Control-Key-c> <Key-F16> <Control-Lock-Key-C> <Meta-Key-w> <Lock-Meta-Key-W> <Control-Key-Insert>	<Key-XF86Copy>
<<Paste>>	<Control-Key-v> <Key-F18> <Control-Lock-Key-V> <Control-Key-y> <Control-Lock-Key-Y> <Shift-Key-Insert>	<Key-XF86Paste>
<<PasteSelection>>	<ButtonRelease-2>	as 8.6

`<<PasteSelection>>` pastes PRIMARY, not CLIPBOARD. To stop a text widget from pasting with the middle button, delete the binding for this one class:

```
bind Text <<PasteSelection>> {break}
```

10.5 Summary

- Mouse wheel: Tk 8.6 under X11 `<Button-4>/<Button-5>`, Tk 9 `<MouseWheel>` everywhere. Under Tk 9, `<Button-4>/<Button-5>` are the side buttons 8 and 9.
- Under X11, `Command` is Alt in 8.6, and no key in 9.0 and 9.1 (container and Xfce). Under `x11`, bind `Control`.
- Super is called `Mod4`.
- PRIMARY by selecting, CLIPBOARD by copying. Both disappear with the program unless a manager takes them over.

Next chapter: Native dialogs and desktop integration

11 Native dialogs and desktop integration

11.1 Introduction

On Windows and macOS, `tk_getOpenFile` calls the system's dialog. On Linux there is no "system" in that sense, but GTK, Qt and the desktop portals, and Tk uses none of them. Tk draws the standard dialogs itself. Desktop integration also includes a `.desktop` file, an icon, notifications and, from Tk 9 on, an icon in the panel and printing via CUPS.

11.2 The standard dialogs are Tk dialogs

```
set datei [tk_getOpenFile -title "Öffnen" \
    -filetypes {{Tcl {.tcl}} {Alle *}}]
set ordner [tk_chooseDirectory -mustexist 1]
tk_messageBox -message "Gespeichert." -type ok
```

On X11, `tk_getOpenFile` leads to `::tk::dialog::file::` (container check, body of the procedure under 8.6.14 and 9.0.4):

```
if {$::tk_strictMotif} {
    return [::tk::MotifFDialog open {*}$args]
} else {
    return [::tk::dialog::file:: open {*}$args]
}
```

The dialog therefore looks the same on every Linux, regardless of Xfce, GNOME or KDE. It knows no bookmarks of the file manager and no network locations from GVfs. `tk_messageBox`, `tk_chooseColor` and `tk fontchooser` are Tk dialogs as well.

A peculiarity of the X11 file dialog: by default it **shows** hidden files (dot files). The source of `tkfbox.tcl` (8.6.14 and 9.0.4) contains `showHiddenVar 1` and `showHiddenBtn 0`. To hide them and offer a toggle, load the dialog once and set both variables:

```
catch {tk_getOpenFile -gibtsnicht}      ;# loads tkfbox.tcl
set ::tk::dialog::file::showHiddenVar 0
set ::tk::dialog::file::showHiddenBtn 1
```

`serie-linux` (howto 11) opens the dialogs only when `SERIES_AUTO` is not set. In the container they were not opened.

If you want the desktop's dialog, call a helper program, such as `zenity --file-selection` (GTK) or `kdialog --getopenfilename` (KDE), via `exec` and read the output. Both were missing in the container. On the workstation (Xfce) `zenity` 4.2.2 is installed, `kdialog` is missing. `zenity --file-selection` from `wish` 8.6.18, 9.0.4 and 9.1.0, identical:

Call	File chosen	Cancel
exec zenity …	catch 1, result path plus GTK warning, -errorcode NONE	catch 1, CHILDSTATUS … 1
exec -ignorestderr zenity …	catch 0, result only the path	catch 1, child process exited abnormally

GTK writes a warning to stderr on every call (`Gtk-WARNING … Failed to measure available space`). Without `-ignorestderr`, `exec` turns this into an error, even if a file was chosen. Therefore:

```
proc dateiWaehlen {} {
    if {[catch {exec -ignorestderr zenity --file-selection} pfad]} {
        return ""          ;# Cancel: exit 1
    }
    return $pfad
}
```

Check beforehand which helper program is available:

```
foreach p {zenity kdialog} {
    if {[auto_execok $p] ne ""} { set hilfe $p; break }
}
```

11.3 The .desktop file

A program appears in the start menu through a .desktop file according to the Desktop Entry Specification. For one user it lives in `~/.local/share/applications/`, for all users in `/usr/share/applications/`. Template `beispiele/meinapp.desktop`:

```
[Desktop Entry]
Type=Application
Name=MeineApp
Name[de]=Meine Anwendung
Comment=Beispiel aus "Tcl/Tk unter Linux"
Exec=/opt/meineapp/bin/meineapp %F
Icon=meineapp
Terminal=false
Categories=Utility;
StartupWMClass=Meineapp
```

Notes on the keys:

- `Exec` with a full path. A program started from the start menu gets the session's `PATH`, not the one from `~/.bashrc`. A `/opt/tcl9/bin` that is only at the front in the shell is missing there. `%F` passes on selected files.
- `Icon` without extension and without path searches the icon theme, e.g. `~/.local/share/icons/hicolor/48x48/apps/meineapp.png`. A full path works too.

- StartupWMClass connects the running window with the entry, so that the panel shows the right icon. The value is the class from `WM_CLASS`. A program started with `wish9.0 meineapp.tcl` has the class `Meineapp.tcl`; with `-name meineapp` it becomes `Meineapp`. See X11, window managers and Wayland.

Check with `desktop-file-validate` from the package `desktop-file-utils`. `serie-xfce` (howto 07) writes a minimal file (`Type`, `Name`, `Exec=true`, `Terminal`, `Categories`) and checks it with that; under Xfce 4.20 without a message. In the container the tool was missing. The template above, `beispiele/meinapp.desktop`, passes on the workstation without a message, exit 0.

11.4 Window icon

```
image create photo ikon -file [file join $hier meineapp.png]
wm iconphoto . -default ikon
```

`-default` applies to all later toplevels. On X11, Tk puts the image into the property `_NET_WM_ICON`; the panel takes it if it finds no better entry via `StartupWMClass`. In the container the call ran without error; without a panel the icon is not visible. Several sizes in one call (`wm iconphoto. -default ikon16 ikon32 ikon48`) leave the choice to the desktop.

11.5 Notifications

Two ways: the program `notify-send` and, from Tk 9 on, `tk sysnotify`.

`serie-xfce` (howto 06) calls under Xfce 4.20:

```
exec {*}[auto_execok notify-send] -t 1000 -e --app-name meineapp \
    "Titel" "Text"
```

There the service is provided by `xfce4-notifyd`. `notify-send` needs the session D-Bus. In the container without a session it ends with

```
Failed to execute child process "dbus-launch" (No such file or directory)
```

and exit status 1.

tk sysnotify loads `libnotify.so.4` (or older versions) at runtime on X11 via `Tcl_LoadFile`; Tk is not linked against it (source `tkUnixSysNotify.c`). If that succeeds, the command `::tk::sysnotify::_sysnotify` exists, and the message goes to libnotify. If the library is missing, Tk draws its own borderless window at the top right that fades out after a while (`systray.tcl`, procedure `_notifywindow`).

Container check with 9.0.4: `libnotify4` is installed, `_sysnotify` is present. `tk sysnotify Titel Text` returns without an error, but nothing appears, and no window is left behind: without D-Bus, libnotify has nobody to hand the message to. A program learns nothing about this. `tk sysnotify` has not been checked with Xfce.

11.6 Icon in the panel

`tk systray` creates an icon in the system tray on X11. For this, Tk uses the system tray protocol (XEmbed), taken over from the package tktray (source `tkUnixSysTray.c`). Container check with 9.0.4:

```
image create photo ikon -width 16 -height 16
tk systray exists                       ;# 0
tk systray create -image ikon -text Probe
tk systray exists                       ;# 1
```

`exists` reports 1, although no system tray is running in the container. The value only says that `create` has run, not that anybody shows the icon. Tk 8.6 does not have the command ("unknown or ambiguous subcommand "systray""). On the workstation (Xfce, 9.0.4 and 9.1.0) likewise `exists` 1 after `create`; the text from `-text` is the tooltip and does not appear as a window name in the window tree. Whether the Xfce panel shows the icon has not been checked visually.

11.7 Printing

`tk print` exists from Tk 9 on. On X11 it shows its own Tk dialog and prints via the commands `lpstat` and `lpr` of CUPS (source `print.tcl`). A canvas goes to the printer as PostScript, a text widget as text. Container check without CUPS:

```
tk print                    wrong # args: should be "tk print w"
::tk::print::cups           Tcl-Ensemble (lpstat/lpr)
::tk::print::cups getprinters
    Unable to obtain the list of printers. Command "lpstat" not found.
    Please install the CUPS package for your system.
```

The printer list comes from `lpstat -e`, the default printer from `lpstat -d`. Printing via CUPS, PDF routes and label printers are covered in the book *Printing*.

11.8 Summary

- `tk_getOpenFile` and its relatives are Tk dialogs on X11, not GTK or Qt dialogs. Hidden files are visible until `showHiddenVar` is 0.
- `.desktop` in `~/.local/share/applications`, `Exec` with a full path, `StartupWMClass` equal to the class from `WM_CLASS`.
- `notify-send` and `tk sysnotify` need the session D-Bus. `tk sysnotify` stays silent if it is missing.
- `tk systray exists` says nothing about visibility.
- `tk print` on X11: CUPS commands `lpstat` and `lpr`.

Next chapter: Xfce from Tcl

12 Xfce from Tcl

12.1 Introduction

Xfce is the desktop on the author's workstation. A Tcl program talks to it via command-line tools: settings via `xfconf-query`, preferred applications via `exo-open`, messages via `notify-send`. All three need the session D-Bus. This chapter follows `serie-xfce`, checked with Xfce **4.20** under X11 on Debian. There is no Xfce in the container; every value here comes from the series.

12.2 Detecting Xfce

`serie-xfce` (howto 01 and 04):

```
$env(XDG_CURRENT_DESKTOP)        ;# XFCE
$env(XDG_SESSION_TYPE)           ;# x11
$env(XDG_MENU_PREFIX)            ;# xfce-
$env(DBUS_SESSION_BUS_ADDRESS)   ;# unix:path=/run/user/UID/bus
exec xfce4-about --version       ;# … Xfce 4.20
```

`XDG_CURRENT_DESKTOP` can be a colon-separated list (`ubuntu:GNOME`). A check therefore looks for one element:

```
proc desktop {} {
    if {![info exists ::env(XDG_CURRENT_DESKTOP)]} { return "" }
    return [split [string toupper $::env(XDG_CURRENT_DESKTOP)] :]
}
if {"XFCE" in [desktop]} { ... }
```

Without a desktop session the series stops before it checks anything. In the container howto 01 reports:

```
SKIP: serie-xfce braucht XFCE (XDG_CURRENT_DESKTOP=)
```

12.3 xfconf: reading settings

Xfce stores its settings in xfconf, a service on the session D-Bus. It is divided into channels, each channel into properties with paths (`serie-xfce`, howto 02):

```
exec xfconf-query -l                          ;# channels
exec xfconf-query -c xfwm4 -l                 ;# properties
exec xfconf-query -c xfwm4 -p /general/theme
exec xfconf-query -c xfwm4 -p /general/workspace_count
```

Existing channels include `xfwm4`, `thunar`, `xfce4-panel`, `xfce4-session`, `xsettings`. The values (theme, number of workspaces) are up to the user. The series therefore checks only type and length, not a specific value. Without a session every call fails with "Verbindung ist gescheitert".

A reader with a default:

```
proc xfconf {kanal pfad {vorgabe ""}} {
    set p [auto_execok xfconf-query]
    if {$p eq ""} { return $vorgabe }
    if {[catch {exec {*}$p -c $kanal -p $pfad 2>/dev/null} wert]} {
        return $vorgabe
    }
    return $wert
}
set skala [xfconf xsettings /Gdk/WindowScalingFactor 1]
```

The channel `xsettings` matters for Tk: at startup under Xfce, Tk 9 reads exactly `/Gdk/WindowScalingFactor` and, if it is 2, sets the interface to 200 % (source `scaling.tcl`, see Fonts, scaling, themes). Tk does not detect Xfce by `XDG_CURRENT_DESKTOP`, but by `ps -e` showing a process `xfce4-session`. On the workstation `ps -e` finds it from `wish9.0` and `wish9.1`. `WindowScalingFactor` is 1 there,

so everything stays at the value from the display: `tk scaling` 1.332, `scalingPct` 100, under 9.0 `::tk::fontScalingFactor` 1.

12.4 xfconf: writing settings

```
exec xfconf-query -c serie-xfce -p /howto/demo -n -t string -s ok
exec xfconf-query -c serie-xfce -p /howto/demo
exec xfconf-query -c serie-xfce -p /howto -r -R
```

`-n` creates a new property, `-t` names its type, `-r -R` deletes recursively. The series (howto 03) writes only into its own test channel and removes it afterwards. A program writes user settings in `xfwm4` or in the panel only if that is its job.

A channel of its own is a convenient place for the settings of an Xfce program, but it ties the program to Xfce. For a program that also runs under other desktops, a file under `XDG_CONFIG_HOME` (see Paths, environment, XDG) is the better choice.

12.5 Preferred applications: exo-open

Under Xfce, `xdg-open` asks `exo-open`. Called directly, `exo-open` starts the applications set in Xfce by role (`serie-xfce`, howto 05 and 09):

```
exec {*}[auto_execok exo-open] --launch FileManager &
exec {*}[auto_execok exo-open] --launch TerminalEmulator &
exec {*}[auto_execok exo-open] --launch WebBrowser https://www.tcl-lang.org &
```

Under Xfce 4.20: the file manager is Thunar 4.20, `xdg-mime query default inode/directory` returns `thunar.desktop`. Text files are opened by `org.xfce.mousepad.desktop`. The terminal is `xfce4-terminal` 1.2.0.

A folder in the file manager:

```
exec {*}[auto_execok xdg-open] $ordner &
```

Thunar directly: `exec thunar $ordner &`. Under `SERIES_AUTO` the series opens no window and only checks whether the tools are present.

12.6 Notifications

The service is provided by `xfce4-notifyd`. On the workstation it lives in `/usr/lib/x86_64-linux-gnu/xfce4/notifyd/`, so not in the `PATH`. `auto_execok xfce4-notifyd` does not find it; whether the service is running is asked via `notify-send` itself or via D-Bus. The call of `notify-send` and the behavior of `tk sysnotify` are described in Native dialogs and desktop integration.

12.7 Window manager, panel, session

`serie-xfce` (howto 10 and 11):

```
exec xfwm4 --version
exec xfce4-panel --version
exec xfce4-session --version
```

xfwm4 is the window manager of the session. It reparents: on the workstation the title bar is 29 pixels high (`serie-xvfb`, howto 04), measured as `winfo rooty. - gy`. With a different theme the value changes. A Tk program under Xfce gets focus, iconify and fullscreen from the window manager; under Xvfb without a window manager all of that is missing, see X11, window managers and Wayland.

The panel layout lives in the channel `xfce4-panel`. The series does not read it.

12.8 Summary

- Detecting Xfce: `XFCE` in `XDG_CURRENT_DESKTOP`, read as a list with `:`.
- Settings: `xfconf-query -c Kanal -p Pfad`. Write only into channels of your own.
- Under Xfce, Tk 9 reads `xsettings /Gdk/WindowScalingFactor`.
- Applications by role: `exo-open --launch`. Folders: Thunar.
- Everything depends on the session D-Bus. Without it every call fails.

Next chapter: GUI tests with Xvfb

13 GUI tests with Xvfb

13.1 Introduction

Tk tests need a display, even on a server without a screen or in a CI pipeline. On Linux, Xvfb provides it, an X server that draws only into memory. Xvfb has no window manager. What that means for focus, frames and iconify is shown in the next chapter. This chapter sets up the display: Xvfb and `xvfb-run`, screen size and DPI, fonts, and the exit status a pipeline can rely on.

The basis is `serie-xvfb`, checked with Tcl/Tk 8.6.14 and 9.0.4 under Xvfb without a window manager and with openbox. In the container all 20 howtos ran with both versions: 19 OK, 1 SKIP (howto 10, iconify, needs a window manager), 0 FAIL.

13.2 Xvfb and xvfb-run

Debian package `xvfb`. It brings the server `Xvfb` and the helper script xvfb-run (`serie-xvfb`, howto 17). If it is missing, a test suite reports SKIP, not FAIL:

```
if {[auto_execok xvfb-run] eq ""} {
    puts "SKIP: xvfb-run fehlt"
    exit 0
}
```

`xvfb-run` starts a server, sets `DISPLAY`, starts the program and stops the server afterwards (`serie-xvfb`, howto 18):

```
xvfb-run -a wish9.0 test.tcl
xvfb-run -a -s "-screen 0 800x600x16 -dpi 50" wish9.0 test.tcl
xvfb-run -a sh -c 'openbox >/dev/null 2>&1 & sleep 1; wish9.0 test.tcl'
```

- `-a` looks for a free display, starting at `:99`.
- `-s "…"` passes options to Xvfb: `-screen 0` WxHxD (width, height, color depth) and `-dpi N`.
- Without `-s`, the `xvfb-run` of the test machine used `-screen 0 1280x1024x24`. Older

versions used `640x480x8`. So always specify it.
- A window manager starts beforehand in the same `sh -c`. `sleep 1` gives it time to register. openbox is the Debian package `openbox`.

A script that does not end by itself keeps `xvfb-run` open. Test scripts end with `exit`; the series enforces this via `SERIES_AUTO=1`.

`beispiele/gui-test-xvfb.sh` sums this up: fixed size, fixed DPI, window manager via `WM=openbox`, SKIP without `xvfb-run`.

13.2.1 Own server instead of xvfb-run

For many short runs a persistent server pays off:

```
Xvfb :93 -screen 0 1280x1024x24 -nolisten tcp -noreset &
export DISPLAY=:93
```

The option -noreset is important. Without it, Xvfb resets its state as soon as the last client closes its connection. In the container this lost every resource loaded with `xrdb` before the test started; `xprop -root` no longer showed a property that had just been set. With `-noreset` they stayed.

13.3 Screen, DPI and fonts

`winfo screenwidth`, `screenheight` and `screendepth` return exactly the values from `-screen` (`serie-xvfb`, howto 01). `tk scaling` follows `-dpi`: 1.389 without the option (Xvfb then reports 100 dpi), 0.695 with `-dpi 50`, 2.770 with `-dpi 200`.

The fonts come via fontconfig from the machine the test runs on, not from Xvfb. Their size is given in points, so the pixel size grows with `-dpi`. A test that expects fixed pixels (width of a label, click point on a tab) breaks with a different DPI or on a machine with a different default font. Such tests calculate relatively:

```
set breite [font measure TkDefaultFont "Hallo Welt"]
set soll   [winfo reqwidth .l]
```

Under Tk 9, the pixel dimensions of ttk also grow with the DPI (in the container: scrollbar 15 pixels at 100 dpi, 27 at 192 dpi). A test that is green under 8.6 can break at the same place under 9, see Fonts, scaling, themes.

For reproducible runs, size, depth and DPI therefore belong fixed in the test call, and the font fixed in the test:

```
font configure TkDefaultFont -family "DejaVu Sans" -size 10
```

13.4 Exit status and abort

A `wish` without a display ends with 0 after running the script without Tk (container check, see X11, window managers and Wayland). For a pipeline this is dangerous: a wrongly set `DISPLAY` looks like a passed test. Test scripts therefore start with:

```
if {[catch {package require Tk} fehler]} {
    puts stderr "FAIL: $fehler"
    exit 1
}
```

`tcltest` does not set the exit status by itself either. In the version from Tcl 9.0.4, `tcltest::runAllTests` returns the value 1 if a test file fails (source `tcltest.tcl`). `all.tcl` passes it on:

```
exit [tcltest::runAllTests]
```

The details are in the book *Testen*.

For hangs, an outer `timeout` helps:

```
timeout 60 xvfb-run -a wish9.0 tests/all.tcl
```

13.5 tclsh or wish

`wish` loads Tk at startup, `tclsh` only with `package require Tk`. After that both behave the same (`serie-xvfb`, howto 14): `.` exists, but while the script runs it is **not yet mapped**; `winfo ismapped .` is 0. Mapping happens only at the next `update` or when the event loop runs.

Under `tclsh` the event loop does not run by itself. A test under `tclsh` needs `update` or `vwait`, a program under `tclsh` a `vwait forever` at the end. `serie-xvfb` chooses the interpreter via the first line of the howto: if it contains `wish`, `run-all.tcl` starts the `wish` matching the version of the running `tclsh`.

13.6 Summary

- Debian package `xvfb`, `xvfb-run -a -s "-screen 0 WxHxD -dpi N"`.
- Without `xvfb-run`: SKIP, not FAIL.
- Own Xvfb with `-noreset`, otherwise it loses resources.
- DPI determines `tk scaling`, fonts come from the client. Fix size, DPI and font in the test.
- `wish` without a display ends with 0. Tests check `package require Tk` themselves and end with an exit status.

Next chapter: What the test sets up itself

14 What the test sets up itself

14.1 Introduction

On the workstation, the window manager gives a new window focus, puts it into a frame and can iconify it. Under Xvfb without a window manager nobody does that. A test that relies on any of these is green on the workstation and red in the pipeline, or the other way round. The rule from `serie-xvfb` (howto 16): what the environment contributes, the test must not assume. It sets it up itself, with `update`, `focus -force` and `winfo rooty`. Then it behaves the same under Xvfb without a window manager and with openbox.

All statements in this chapter are checked in `serie-xvfb` with Tcl/Tk 8.6.14 and 9.0.4, without a window manager and with openbox. The values without a window manager were confirmed by the container check with `beispiele/wm-probe.tcl`. The exception is the Xfce case in the section *The frame comes late*; it comes from test runs on a workstation.

14.2 Overview

	Xvfb without window manager	Xvfb with openbox
winfo manager .	wm	wm
wm frame . vs. winfo id .	equal	different
winfo ismapped, winfo viewable after update	1	1
focus -force	works	works
Focus after update without focus -force	empty	.
Title bar rooty - gy	0	20
wm iconify	stays normal, about 2 s	iconic
local grab	works	works

14.3 Mapping: update first

A window that the script has just created is not yet on the screen. It is mapped at the next `update` (`serie-xvfb`, howto 03 and 14):

```
toplevel .t
winfo ismapped .t        ;# 0
update
winfo ismapped .t        ;# 1, even without a window manager
```

winfo viewable additionally requires that all ancestors are mapped. After `wm withdraw .`, a frame in `.` is not `viewable`, even if it is mapped itself (howto 15).

14.4 Generating keys: establishing focus

`event generate` with a key event goes to the widget with the **keyboard focus**, not to the window you name (howto 07):

```
entry .e1; entry .e2; pack .e1 .e2
update
focus -force .e1
update
event generate .e2 <Key-x>      ;# ends up at .e1
```

Three pitfalls follow from this, all checked without a window manager and with openbox:

1. **Before update:** a new toplevel is not mapped and has no focus. `event generate .t <Escape>` does not arrive (howto 05). This is not an Xvfb difference, it applies everywhere.
2. **Withdrawn:** a window after `wm withdraw` does not get the key, not even after `focus -force .`. Only after `wm deiconify`, `update` and `focus -force` (howto 06).
3. **Without a window manager:** after `update`, `focus` is empty. Nobody assigns the focus. A generated key arrives only after `focus -force` (howto 07 and 20). With openbox, `.` usually already has the focus after `update`. The test still does not wait for it.

The pattern for every key test is in the series as `serie::fokusVorbereiten`:

```
proc fokusVorbereiten {{w .}} {
    update
    focus -force $w
    update
}
fokusVorbereiten .
event generate . <Control-s>
```

Container check with `beispiele/wm-probe.tcl`: key without `focus -force` arrived 0 times, with `focus -force` once, under 8.6.14 and 9.0.4.

14.5 Grab

A local grab is internal to Tk and needs no window manager (howto 11). The series makes the window visible first:

```
toplevel .dlg
update
grab .dlg
grab current            ;# .dlg
```

Without `update` no error occurs. Container check, 8.6.14 and 9.0.4: `grab .d` on a freshly created, not yet visible toplevel returns without error, `grab current` is `.d`, `grab status .d` is `local`; the same after `wm withdraw .d`. A local grab on an invisible window thus silently locks the rest of the interface. A global grab (`grab -global`) depends on the X server. On the workstation (Xfce, 8.6.18, 9.0.4, 9.1.0) it returns without error, `grab status` is `global`, `grab release` lifts it; the session remained usable. In a test only with `grab release` in `finally`.

14.6 Position of dialogs

`wm geometry +x+y` sets the **frame corner** of a window, `winfo rootx/rooty` returns the corner of the **client area** (howto 04 and 12). A dialog that is to sit directly below the title bar of the main window:

```
set x [winfo rootx .]
set y [winfo rooty .]
wm geometry .dlg +$x+$y
```

The dialog's frame then begins at the top edge of the main window's client area; its own client area lies lower by its own title bar. Without a window manager, `winfo rooty .dlg` equals `y`, under openbox `y + 20`.

A fixed number is always wrong somewhere (howto 13). Measured title bars: without a window manager 0, openbox 20, Xfce on the author's workstation 29. Code like

```
wm geometry .dlg +[expr {$gx}]+[expr {$gy + 80}]
```

is only right if the title bar happens to be 80 pixels. The series demo measures the frame and checks that 80 would be wrong here.

`scan [wm geometry.] {%dx%d%[+-]%d%[+-]%d}` also reads the form `-X-Y`. Then X counts

from the right, and the calculation above does not hold. Set it with `+X+Y` beforehand.

14.6.1 The frame comes late

When the window manager adds its frame depends on the window manager. Under Xvfb with openbox, for a simple toplevel it is already there when the first `update` after `wm geometry` returns; this also holds for a transient one. A probe script with a dialog at `+300+250` gave with 8.6.14 and 9.0.4:

Time	without window manager	openbox
after update	winfo rootx/rooty 300,250	301,270
10 to 1000 ms later	unchanged	unchanged
<Configure> with offset	none	before <Map>, after 17 to 28 ms

On an Xfce workstation with wish 9.1.0 the offset came later. A dialog that the code had corrected once after 100 ms ended up 5 pixels to the right and 28 to 29 pixels below the target. This did not happen in every run: in both runs of the whole suite, and in the single run of the test file in the third of three. Under Xvfb, with and without openbox, it never happened.

For the code this means: a correction after a fixed wait is not enough. It watches `<Configure>` on the dialog for a while, about one second after it appears, and corrects each time shortly afterwards. It calculates from the frame corner from `wm geometry`, not from `winfo rootx`; otherwise the correction overshoots the target when there is no frame.

For the test this means: under Xvfb the case does not occur by itself. The test sets it up and moves the dialog itself after it appears, starting from the frame corner:

```
after 300 {
    regexp {\+(-?\d+)\+(-?\d+)$} [wm geometry .d] -> fx fy
    wm geometry .d +[expr {$fx + 5}]+[expr {$fy + 29}]
}
```

Using `winfo rootx` here instead of the frame corner additionally shifts by the frame under openbox, in the probe run 6 and 49 instead of 5 and 29 pixels. With the centered transient dialog from the next section, the frame corner also gives 6 and 49, because `wm geometry` is not correct there. That does no harm to a test that only checks that the correction takes effect; one that compares the offset exactly places the dialog beforehand not exactly centered or not transient.

`beispiele/spaeter-rahmen.tcl` shows both side by side. With 8.6.14 and 9.0.4, without a window manager and with openbox, the result was the same, likewise on the Debian workstation under Xvfb without a window manager with 8.6.18, 9.0.4 and 9.1.0:

```
einmal     Abstand der Mitten: 5,29
beobachten Abstand der Mitten: 0,0
```

14.6.2 openbox places centered dialogs itself

An exception showed up under openbox with a transient dialog that the program places exactly centered over the client area of the parent window. Parent window `.p` with `wm geometry .p 800x500+500+300`, dialog 418×92 pixels, transient to `.p`; read 50 ms after placing, with 8.6.14 and 9.0.4:

requested frame corner	wm geometry	frame according to xwininfo	client area
+692+524 (centered)	+692+524	691,504	692,524
+680+524	+680+524	680,524	681,544
+692+524, not transient	+692+524	not read	693,544

In the first case the frame lies centered over the frame of `.p`, not at the requested position. Apparently openbox places a transient dialog that is requested centered in the center itself. The client area is then immediately at the target. But `wm geometry` keeps reporting the requested position, still after 950 ms; the frame corner derived from it is off by 1 and 20 pixels. Only after the program sets the position again itself do the values match again.

14.7 Tests that need a window manager

Iconify, real frame position and focus changes by the window manager cannot be checked without one. Such tests report SKIP if none is running. The detection (howto 09) is described in X11, window managers and Wayland: `wm frame` vs. `winfo id`, otherwise an iconify probe.

In the series the guards are called `serie::brauchtWM` (any window manager) and `serie::brauchtRahmen` (a reparenting one). Without a window manager the iconify probe costs about two seconds, because Tk waits for a state change (container: 2002 and 2003 ms). The series therefore lets `SERIE_WM` preset the result:

```
SERIE_WM=0 xvfb-run -a wish9.0 run-all.tcl
```

For tests with a window manager, `xvfb-run` starts openbox beforehand, see GUI tests with Xvfb. The howtos have not been run with xfwm4 under Xvfb.

On Windows and macOS there is always a window manager. There nothing skips; the X11-specific howtos report SKIP because `tk windowingsystem` is not `x11`. The howtos have not been tried on Windows.

14.8 Summary

- `update` before every check of visibility, position or focus.
- Generated keys go to the focus widget. `focus -force`, then `update`, then `event generate`.
- A local `grab` only needs a visible window.
- Dialog position from `winfo rooty`, never from `wm geometry` plus a fixed number.
- A frame can arrive after mapping. Watch `<Configure>` for a while; in the test, produce the late shift yourself.
- openbox places a transient dialog requested centered itself; `wm geometry` then does not report the frame corner.
- Iconify and frames only with a window manager; without one SKIP, preset via `SERIE_WM`.

Next chapter: Building Tk: dependencies

15 Building Tk: dependencies

15.1 Introduction

Building Tcl/Tk 9 on Linux yourself requires a compiler and the X11 development packages. The build itself, with all switches, is described in *Tcl/Tk 9.0 bauen* and *Tcl/Tk 9.1 bauen*. This chapter only checks which libraries a Linux Tk actually needs and brings along, and where the Debian package differs from a source build. It is based on `serie-tcltk-bauen`, checked on Debian forky (September 2026) with `tclsh8.6` (8.6.18) and `tclsh9.0` (9.0.4), 12 of 12 howtos passed in each case. In the container the series ran under 8.6.14 with 11 OK and 1 SKIP (howto 10: Ubuntu does not have Itcl installed for 8.6), and under 9.0.4 with 11 OK and 1 FAIL; the FAIL is described below.

15.2 What Debian needs to build Tk

`apt-cache showsrc tk8.6` lists four X development packages and `tcl8.6-dev` as Build-Depends (howto 02):

```
libx11-dev libxss-dev libxext-dev libxft-dev tcl8.6-dev
```

Plus the toolchain from `build-essential`. `libxft-dev` pulls in fontconfig and freetype. For freetype the package is now called `libfreetype-dev`; `libfreetype6-dev` is only a transitional package.

According to *Tcl/Tk 9.0 bauen* (tried on Ubuntu 24.04), a source build of Tk 9 additionally needs libxi-dev: `configure` got stuck there on `X11/extensions/XInput2.h`. Debian does not list `libxi-dev` for `tk8.6`, and no linked Tk library needs `libXi` (see below). `XInput2` does not occur in the source code of Tk 9.0.4 (container search across all files). Where the requirement came from is not resolved. The series' recommendation stands: for a self-built Tk 9, install the package; it does no harm, and `configure` does not abort late for lack of the header.

15.3 TK_LIBS and TCL_LIBS

What an installation links is recorded in `tkConfig.sh` and `tclConfig.sh` (howtos 03 and 04). Debian forky:

```
TK_LIBS  (8.6)  -lX11 -lXss -lXext -lXft -lfontconfig -lfreetype … -lz …
TCL_LIBS (8.6)  -ldl -lz -lpthread -lm
TCL_LIBS (9.0)  -ldl -lz -lpthread -lm -ltommath
```

Container check:

```
TK_LIBS  (Quellbau 9.0.4)  -lX11 -lXss -lXext -lXft -lfontconfig -lfreetype
    -lfontconfig -ldl -lz -lpthread -lm
TCL_LIBS (Ubuntu 8.6.14)   -ldl -lz -lpthread -lm
TCL_LIBS (Quellbau 9.0.4)  -ldl -lz -lpthread -lm
```

This is where the FAIL in the container comes from: howto 03 expects `-ltommath` under 9.0. The Debian package `tcl9.0` links against the system library libtommath; a source build compiles in the bundled version and needs no `-ltommath`. Both are correct; the demo only checks the Debian situation.

`zlib1g-dev` is mandatory for Tcl. If it is missing, a self-build often stops with an unclear `configure` message (howto 03).

15.4 What is really linked

TK_LIBS is the intention, `ldd` on the library is the result (howto 07). Debian forky, `libtk8.6.so` and `libtcl9tk9.0.so`: Xft, Xss, X11, Xext, fontconfig, freetype, Xrender, zlib, no `libXi`. `ldd` lists `libpng16` for all three Tk versions, but only via freetype (see below).

`ldd` also shows what a library pulls in through others. Xrender comes via Xft, not as its own `-lXrender`. The list of direct dependencies is more precise:

```
readelf -d /opt/tcl9/lib/libtcl9tk9.0.so | grep -E 'NEEDED|RUNPATH'
```

Container check:

Library	directly needed (NEEDED)	RUNPATH
libtk8.6.so (Ubuntu)	Xft, fontconfig, X11, Xss, m, c	none
libtcl9tk9.0.so (source build)	Xft, fontconfig, X11, Xss, m, c	/opt/tcl9/lib
libtcl8.6.so (Ubuntu)	z, m, c	none
libtcl9.0.so (source build)	z, m, c	/opt/tcl9/lib

In the container `ldd` lists `libpng16` for **both** Tk versions, as a dependency of freetype. The same on the workstation (Debian forky): `readelf -d` shows no `libpng` under `NEEDED` for `libtk8.6.so`, `libtcl9tk9.0.so` and `libtcl9tk9.1.so`; `libfreetype.so.6` has it directly there. Tk 9.1 additionally needs HarfBuzz and Xrender directly.

The `RUNPATH /opt/tcl9/lib` is set by the source build. It ensures that `tclsh9.0` finds its library without `LD_LIBRARY_PATH`, but ties it to this location. This becomes important when distributing, see Distributing.

15.5 tclConfig.sh on Debian

On Debian and Ubuntu, `/usr/lib/tcl8.6/tclConfig.sh` is a two-line wrapper (howto 09; the same in the container):

```
#! /bin/sh
. /usr/lib/`dpkg-architecture -qDEB_HOST_MULTIARCH`/tcl8.6/tclConfig.sh
```

The values are in `/usr/lib/x86_64-linux-gnu/tcl8.6/tclConfig.sh`. Searching the wrapper file for `TCL_LIBS` with `grep` finds nothing. A source build into `/opt/tcl9` puts `tclConfig.sh` directly into `/opt/tcl9/lib`; `configure --with-tcl=` wants exactly this directory. For TEA extensions see *Tcl-Erweiterungen*.

15.6 Threads and shared

- Tcl 8.6 is always built with threads: `TCL_THREADS=1`. Tcl 9 no longer has a switch; `--enable-threads` only produces `WARNING: unrecognized options` (howto 05).
- A normal Linux build is shared: `TCL_SHARED_BUILD=1`, `libtcl9.0.so`, `info sharedlibextension` is `.so` (howto 06; container: all three confirmed). Static (`--disable-shared`) is the exception for zipkits.

15.7 Two pitfalls when building

From `serie-tcltk-bauen`, howto 11:

- **Empty .o:** An aborted `make` leaves behind object files of zero bytes with a new timestamp. `make` does not compile them again, and the linker later fails with `file format not recognized`. Find them with `find. -name '*.o' -size 0`.
- **LD_LIBRARY_PATH with Tcl 9:** The freshly built `tclsh` in the build directory does not find `libtcl9.0.so` as long as it is not installed. The message suggests `TCLSH_NATIVE`; what is meant is the library path: `env LD_LIBRARY_PATH=$PWD make`.

The series' `configure` lines (howto 12):

```
cd tcl9.0.4/unix
./configure --prefix=/opt/tcl90 --enable-shared --enable-64bit
env LD_LIBRARY_PATH=$PWD make
make install

cd ../../tk9.0.4/unix
./configure --prefix=/opt/tcl90 --exec-prefix=/opt/tcl90 \
    --with-tcl=/opt/tcl90/lib --enable-64bit
env LD_LIBRARY_PATH=/opt/tcl90/lib make
make install
```

`configure` runs in the `unix/` directory, not in the root of the tree.

15.8 Summary

- Debian builds Tk against `libx11-dev libxss-dev libxext-dev libxft-dev`. For a Tk 9 source build, add `libxi-dev`.
- Linked are Xft, fontconfig, X11, Xss; through these Xext, Xrender, freetype, zlib, `libpng`. No `libXi`. Tk 9.1 also HarfBuzz and Xrender directly.
- `-ltommath` only with the Debian package `tcl9.0`, not with a source build.
- `tclConfig.sh` on Debian is a wrapper; the values are in the multiarch file.
- A source build sets `RUNPATH` to the prefix.

Next chapter: Distributing

16 Distributing

16.1 Introduction

A Tcl program leaves its own machine in one of three ways: as a script for the Tcl on the target machine, as a system package (`.deb`, `.rpm`) or as a single executable file with zipfs. The book *Ausliefern* describes all three in detail, with zipkits, starkits and installers. This chapter only names what stands out on Linux: which Tcl the target machine has, what an image depends on, and what is needed for the desktop. The basis for zipfs is `serie-tcl9-app`, rerun completely in the container (18 of 18 OK, `tclsh9.0` 9.0.4).

16.2 Which Tcl does the target machine have?

A script needs a suitable Tcl on the target machine. On Linux this is usually 8.6 from the

package:

- Debian Stable (trixie): Tcl 8.6.16 and the package `tcl9.0` 9.0.1.
- Debian Testing (forky) and unstable: 8.6.18, `tcl9.0` 9.0.4 and `tcl9.1` 9.1.0.
- Ubuntu 24.04: 8.6.14. In the container `apt-cache policy` knows no package `tcl9.0`.

A program that requires Tcl 9 therefore does not run with the system Tcl on most Linux machines. It either brings its own Tcl, or it stays 8.6-capable. For a `.deb` with `Depends: tcl9.0`: the package must exist on the target system. *Ausliefern*, chapter *Linux: deb und rpm*, therefore advises checking `apt-cache policy tcl9.0 tcl8.6` on the target beforehand.

A script that is to run under both versions observes the differences in this book: tilde (see Paths, environment, XDG), encoding of scripts (Encoding and locale), mouse wheel (Mouse, keyboard, clipboard).

16.3 An image with zipfs mkimg

Tcl 9 can attach a directory as a ZIP to an interpreter. The result is an executable file that mounts its archive under `//zipfs:/app` at startup and runs `main.tcl` from it (`serie-tcl9-app`, howtos 06 and 07):

```
zipfs mkimg app.bin app.vfs app.vfs "" [info nameofexecutable]
file attributes app.bin -permissions 0o755
```

The fourth argument `""` is the password. It must be present when the base follows as the fifth, otherwise the path ends up in the password field.

Container check with a `main.tcl` that prints its location, built on `/opt/tcl9/bin/tclsh9.0`:

```
Größe     16374 Byte
lib       //zipfs:/lib/tcl/tcl_library
mounts    //zipfs:/lib/tcl  /opt/tcl9/lib/libtcl9.0.so
          //zipfs:/app      …/app.bin
script    //zipfs:/app/main.tcl
```

16 KB, because the image contains only the small `tclsh9.0` and its own archive. The Tcl library is inside `libtcl9.0.so` and is mounted from there. The image therefore depends on exactly this file:

```
ldd app.bin | grep tcl
#   libtcl9.0.so => /opt/tcl9/lib/libtcl9.0.so
readelf -d /opt/tcl9/bin/tclsh9.0 | grep RUNPATH
#   Library runpath: [/opt/tcl9/lib]
```

On a machine without `/opt/tcl9/lib/libtcl9.0.so` it does not start. This has not been checked on a second machine, but follows from `RUNPATH` and `NEEDED`. Such an image is good for practice, not for distributing.

`serie-tcl9-app` stresses the opposite for static bases: if the Tcl library is in the archive of the executable itself, `mkimg` replaces this archive, and `tcl_library` must be copied into your own `app.vfs` (howto 15). With the dynamic source build under `/opt/tcl9` it is in `libtcl9.0.so` and is preserved. Which situation applies is shown by `zipfs mount` in the base.

For GUI programs the base is `wish9.0`, not `tclsh9.0` (howto 09). The image then additionally

depends on `libtcl9tk9.0.so` and on the X libraries of the target machine (see Building Tk: dependencies).

16.4 Zipkit: static base

To distribute, the image needs a base that does not point to `/opt/tcl9`: a statically built `tclsh` or `wish` with an attached Tcl library, a zipkit (`serie-tcl9-app`, howto 13). How to build or obtain one (BAWT, your own build with `--disable-shared`) is described in *Ausliefern*, chapter *Zipkits bauen und füllen*, and in *Tcl/Tk 9.0 bauen*, chapter *Statisch bauen*.

Even a static Tk depends on the system's X libraries (`libX11`, `libXft`, `libfontconfig`). These exist on every Linux desktop, but not on every server. And it depends on the C library: the source build in the container requires symbols up to `GLIBC_2.38` (`objdump -T`), Ubuntu 24.04 has 2.39. On a target with an older glibc it does not start. Anyone building for many distributions builds on the oldest. This has not been checked.

Because an image is just base plus ZIP, the same ZIP content can be combined with a different base for a different platform, as long as it contains no `.so` (howtos 16 and 17):

```
cat zipkit-linux-x86_64 app.zip > app
chmod +x app
```

16.5 Desktop: launcher and icon

A program for the desktop needs, besides the executable file:

- a `.desktop` file in `~/.local/share/applications/` or `/usr/share/applications/`,
- an icon in the icon theme, for example `/usr/share/icons/hicolor/48x48/apps/meineapp.png`,
- a window class that matches `StartupWMClass`.

All three are described in Native dialogs and desktop integration. With an image, Tk takes the name not from the file but from the start script in the archive. Container check, image `meineapp.bin` based on `wish9.0`: `tk appname` is `main.tcl`, the class `Main.tcl`. Every image with `main.tcl` thus has the same name for the window manager. With `./meineapp.bin -name meineapp` it becomes `Meineapp`. The `-name` therefore belongs in the `Exec` line. With a `wish` base, Tk is already loaded when `main.tcl` runs; there only the command line helps. With a `tclsh` base, `main.tcl` can set it in `::argv` before `package require Tk` (see X11, window managers and Wayland).

16.6 Summary

- The target Tcl is usually 8.6. Tcl 9 is available as a package in Debian Stable (`tcl9.0` 9.0.1), Testing and unstable (`tcl9.0` 9.0.4, `tcl9.1` 9.1.0); Ubuntu 24.04 does not have it.
- `zipfs mkimg` on a dynamic `tclsh9.0` yields a small image that depends on `/opt/tcl9/lib/libtcl9.0.so`. To distribute, use a static base (zipkit).
- Even when static, X libraries and glibc are up to the target machine.
- The desktop needs `.desktop`, an icon and a matching window class.
- Details: *Ausliefern*.

Next chapter: FAQ

17 FAQ

Short answers with a reference to the chapter that gives the reasons.

17.1 Platform and interpreter

How do I detect Linux in code?
`$tcl_platform(os) eq "Linux"`. `platform` is `unix` on macOS too. See tcl_platform and the distribution.

How do I detect Debian or Ubuntu?
Read `/etc/os-release`, check `ID` and `ID_LIKE`. `/etc/debian_version` exists on Ubuntu too.

tclsh9.0 does not find Tcllib, but tclsh8.6 does. Why?
A Tcl 9 under `/opt` does not have `/usr/share/tcltk` in `auto_path`. Add pure Tcl packages selectively, not via `TCLLIBPATH` for all versions. See Search paths.

wish starts Tk 8.6 although /opt/tcl9/bin is first in PATH.
A source build creates `wish9.0`, but no `wish` link. Call it with the version number. See Which Tcl?.

file normalize ~ returns a path in the current directory under Tcl 9.
Tcl 9 no longer expands the tilde. Use `$env(HOME)` or `file home`. See Paths, environment, XDG.

Where do my program's settings go?
`$XDG_CONFIG_HOME/<programm>/`, otherwise `~/.config/<programm>/`.

17.2 Encoding

From cron or systemd, umlauts arrive as Ã¼.
The locale is missing there, `encoding system` is iso8859-1. Tcl 8.6 then also reads the script wrongly. Open files with `fconfigure -encoding utf-8`, and for 8.6 put the guard at the start of the script. See Encoding and locale.

Tcl 9 aborts when reading an old file.
The file is not UTF-8. Tcl 8.6 masked this. Specify the correct encoding, or read with `-profile replace`.

17.3 Processes

How do I open a file or address with the default program?
`exec {*}[auto_execok xdg-open] $ziel &`. Under Xfce this goes to `exo-open`. See exec, xdg-open, processes.

exec reports an error although the program ran.
It wrote to stderr. `errorCode` is then `NONE`. `-ignorestderr` or `2>@1`.

My program hangs when reading the clipboard with xclip.
It is itself the owner of the selection and blocks in `exec`. Read through a channel with `fileevent`, or ask Tk directly: `selection get`.

17.4 User interface

Does Tk run under Wayland?

Via XWayland. `tk windowingsystem` remains `x11`. Under Weston, window positions there can be neither set nor read; a real Wayland session has not been checked. See X11, window managers and Wayland.

Under Tk 9 my canvas no longer scrolls with the wheel, but with the mouse's side buttons.

Tk 9 reports the wheel as `<MouseWheel>`. `<Button-4>` and `<Button-5>` are buttons 8 and 9 there. See Mouse, keyboard, clipboard.

<Command-c> from my Mac code does nothing on Linux.

Under Tk 9, `Command` on X11 is `Meta`; in the container and on the Xfce workstation no key was assigned to it. Under Tk 8.6 it was Alt. Under `x11`, bind `Control`.

The clipboard is empty after my program has exited.

On X11 the program holds the content itself. Without a clipboard manager it is gone with the program.

On the 4K screen everything is tiny.

`tk scaling` follows the DPI of the X server. Tk 9 additionally reads the scaling of Xfce, MATE, GNOME and `Xft.dpi`. Under 8.6 set `tk scaling` by hand. See Fonts, scaling, themes.

On one machine my program suddenly has yellow buttons.

X resources. Check `xrdb -query` and `~/.Xdefaults`.

Why doesn't the file dialog look like in GTK programs?

On X11 Tk draws it itself. See Native dialogs and desktop integration.

The panel shows a wrong icon or no icon at all for my program.

`StartupWMClass` in the `.desktop` file does not match the class. `wish9.0 meineapp.tcl` has the class `Meineapp.tcl`, an image with `main.tcl` the class `Main.tcl`. Set it with `-name`.

17.5 Tests

The CI reports success, but no test ran.

`wish` without a display exits with 0. At the start of the test, check `package require Tk` and `exit 1`. See GUI tests with Xvfb.

Under Xvfb, event generate does not arrive.

Without a window manager nobody assigns the focus. `update`, `focus -force`, `update`. See What the test sets up itself.

wm iconify takes two seconds and changes nothing.

No window manager. Tk waits in vain for the state change.

17.6 Building and distributing

Tk's configure does not find XInput2.h.

Install `libxi-dev`. See Building Tk: dependencies.

My zipfs mkimg program does not start on another machine.

The base was a dynamic `tclsh9.0` and depends on `/opt/tcl9/lib/libtcl9.0.so`. Use

a static base (zipkit). See Distributing.

Next chapter: Pitfalls

18 Pitfalls

Short list. Each line describes what **not** to do. The reasoning is in the chapter named.

18.1 Platform and interpreter

- Detecting Linux with `tcl_platform(platform) eq "unix"`. That also matches macOS. Use `os`.
- Reading the distribution from `tcl_platform(osVersion)`. That is the kernel, in the container even the host's.
- Detecting Debian by `/etc/debian_version`. Ubuntu has the file too (container: `trixie/sid`).
- Reading `tcl_platform(threaded)` without `info exists`. Under Tcl 9 the key is missing.
- Taking `$env(USER)` for the user. It can be missing. `tcl_platform(user)` is reliable.
- Calling `wish` without a version number and expecting Tk 9. A source build does not create a `wish` link.
- Installing your own Tcl into `/usr`. Use `/opt`.
- Comparing permissions from `file attributes -permissions` as strings. 8.6 returns `041777`, 9.0 `0o41777`.
- Taking `[lindex [file volumes] 0]` for `/`. Under Tcl 9 `//zipfs:/` comes first.
- On Debian, installing `tcl9.1` without `tk9.1` and expecting Tk 9.1. `package require Tk` then loads Debian's Tk 9.0.
- Assuming a Tcl under `/opt` is separate without checking with `ldd`. It can load Debian's `libtcl` of the same name.
- Using an alias as the launch file. It does not apply in scripts and makefiles.

18.2 Paths and search paths

- `file normalize ~` under Tcl 9. That yields `./~`. `$env(HOME)` or `file home`.
- Hard-coding `~/Documents`. The directory is localized; ask `xdg-user-dir DOCUMENTS`.
- Storing settings as a dotfile directly in `$HOME`. XDG: `~/.config/<programm>/`.
- Separating `TCLLIBPATH` with colons. It is a Tcl list.
- Separating `TCL9_0_TM_PATH` with spaces. Here the colon applies.
- Setting `TCLLIBPATH` for all versions to a directory with Tcl 9 packages. `tclsh8.6` sees it too.
- Expecting a Tcl 9 under `/opt` to see the packages from `/usr/share/tcltk`.
- Putting your own packages for the system Tcl into `/usr/share/tcltk`. That belongs to `apt`; use `/usr/local/share/tcltk`.
- Expecting `libsqlite3-tcl` up to 3.53.4-2 under Debian's Tcl 9. It is built only for 8.6; only 3.54.1-1 (unstable) supports Tcl 9.
- Placing a second sqlite3 with a lower number alongside. The highest number wins; the directory only counts when the numbers are equal.
- A package as `.tm` and alongside it as `pkgIndex.tcl`. Which copy loads depends on what was loaded before; see *Which copy wins* in the chapter *Search paths*.
- Placing your own sqlite3 for Tcl 9 only in front of Debian's directory. Under 8.6 the index then

displaces Debian's version.
- Taking `package ifneeded` as proof of which library is loaded. Use `info loaded`.
- Testing a package draft with `dpkg -i`. Unpacking with `dpkg-deb -x` and loading via `auto_path` is enough and changes nothing under `/usr`.
- Forgetting the matching `libsqlite3-0` when testing an unpacked `libsqlite3-tcl`. The dynamically linked binding needs the same SQLite version.

18.3 Encoding

- Assuming Linux is always UTF-8. Without a locale, `encoding system` is iso8859-1.
- Opening a file without `fconfigure -encoding`.
- Starting a UTF-8 script with umlauts under Tcl 8.6 from cron without a guard or `-encoding utf-8`.
- Relying on `LANG=de_DE` having the same effect under 8.6 and 9. 8.6: iso8859-1, 9.0: utf-8.
- Setting `LANGUAGE` for `msgcat`. `msgcat` reads `LC_ALL`, `LC_MESSAGES`, `LANG`.

18.4 Processes

- `exec` without `auto_execok` on a program that may be missing.
- Interpreting every `exec` error as a non-zero exit status. A line on stderr is an error too.
- Calling `xdg-open` without `&`.
- Using a blocking `exec` to call a program that reads your own X selection. Both wait for each other.
- Calling `auto_reset` just to make `auto_execok` search again. It also removes commands loaded later.

18.5 X11 and window managers

- Relying on the exit status of `wish`. Without a display it exits with 0.
- Detecting a window manager with `winfo manager .`. That is always `wm`.
- Comparing `wm frame .` and `winfo id .` as strings. 8.6 writes leading zeros.
- Placing a dialog below the main window with `gy + 80`. Frames are 0, 20 or 29 pixels high. `winfo rooty .`.
- Taking `wm geometry` for the position of the client area. It is the frame corner.
- Correcting a dialog's position once after a fixed delay. Some window managers create the frame later.
- Under openbox, taking `wm geometry` of a centered transient dialog for the frame corner. openbox places it itself; Tk reports the requested position.
- Expecting `-fullscreen`, `-zoomed`, `iconify` without a window manager.
- Writing `StartupWMClass=meineapp` and starting `wish meineapp.tcl`. The class is `Meineapp.tcl`.
- Under Wayland, expecting `tk windowingsystem` to be `wayland`. It is `x11`.

18.6 Font, scaling, themes

- Assuming a font family. fontconfig substitutes silently.
- Hard-wiring pixel sizes in tests. They depend on DPI, font and, under Tk 9, on the scaling.
- Under Tk 8.6, assuming `Xft.dpi` affects `tk scaling`. It affects only the font.
- Expecting a native Linux theme. There are `clam alt default classic`.
- Starting Xvfb without `-noreset` and loading resources with `xrdb`. The server forgets them as soon as the last client leaves.

18.7 Mouse, keyboard, clipboard

- Under Tk 9, binding the wheel via `<Button-4>`/`<Button-5>`. Those are the side buttons 8 and 9 there.
- Under Tk 8.6 on X11, binding only `<MouseWheel>`. The wheel arrives there as `<Button-4>`/`<Button-5>`.
- Binding `<Command-c>` on X11. 8.6: Alt, 9.0 and 9.1: no key (container and Xfce).
- Binding `<Super-c>`. The name is `Mod4`.
- Taking selecting for copying. Selecting sets PRIMARY, copying sets CLIPBOARD.
- Expecting the clipboard to survive the program's exit.
- `selection get -selection PRIMARY` without `catch`.

18.8 Desktop

- Expecting the GTK file dialog. Tk draws its own, and it shows hidden files.
- Relying on the shell's `PATH` in `Exec=` of a `.desktop` file.
- Calling `zenity` without `-ignorestderr`. The GTK warning on stderr becomes an error, even if a file was selected.
- Taking `tk systray exists` for "icon visible".
- Taking `xfce4-clipman` installed for "clipboard persists". It must be running.
- Taking `tk sysnotify` for "notification shown". Without D-Bus it stays silent.

18.9 Tests

- A key test without `update` and `focus -force`.
- Sending `event generate .w <Key>` to a widget without focus and expecting the binding of `.w`. The key goes to the focus widget.
- Reporting FAIL without `xvfb-run`. SKIP is correct.
- Calling `xvfb-run` without `-s` and expecting the screen size of another version.
- Forgetting `exit [tcltest::runAllTests]`. Otherwise `all.tcl` exits with 0.

18.10 Building and distributing

- Leaving out `libxi-dev` for a Tk 9 source build.
- Passing `--enable-threads` to Tcl 9. Unknown switch.
- Searching for `TCL_LIBS` in `/usr/lib/tcl8.6/tclConfig.sh`. That is a wrapper.
- Distributing a `zipfs mkimg` built on a dynamic `tclsh9.0`. It depends on `/opt/tcl9/lib`.
- Building an image on a newer glibc than the target has.
- Leaving out the password argument of `zipfs mkimg` when the base follows.
- Installing a TEA package against Debian's Tcl with `make install-binaries`. It wants to go to `/usr/bin`; use `install-lib-binaries`.
- Building an extension for Tcl 9 against 9.1 and expecting it under 9.0. Build against 9.0.

Next chapter: CLI

19 CLI

Quick reference for the commands in this book and the example scripts. Unless stated otherwise, the results come from the container check on 2026-10-20 (Ubuntu 24.04, Tcl/Tk 8.6.14 and 9.0.4, Xvfb without a window manager).

19.1 At runtime

```
array get tcl_platform
info patchlevel
info nameofexecutable
puts $tcl_library
puts $auto_path
tcl::tm::path list
tcl::pkgconfig get libdir,runtime
zipfs mount                         ;# Tcl 9 only
encoding system
tk windowingsystem                  ;# x11
winfo server .
tk scaling
tk::pkgconfig get fontsystem        ;# xft
ttk::style theme names              ;# clam alt default classic
```

19.2 Shell

```
type -a tclsh tclsh8.6 tclsh9.0 wish9.0
dpkg -S /usr/bin/tclsh8.6
cat /etc/os-release
locale -a
echo "$XDG_SESSION_TYPE $DISPLAY $WAYLAND_DISPLAY $XDG_CURRENT_DESKTOP"
xdg-user-dir DOCUMENTS
xdg-mime query default inode/directory
fc-match sans-serif
xrdb -query
xmodmap -pm
xprop WM_CLASS                      # then click into the window
readelf -d /opt/tcl9/lib/libtcl9tk9.0.so | grep -E 'NEEDED|RUNPATH'
xvfb-run -a -s "-screen 0 1280x1024x24 -dpi 96" wish9.0 test.tcl
xfconf-query -c xsettings -p /Gdk/WindowScalingFactor
```

19.3 Check script

`beispiele/messprobe-linux.tcl` prints all values used by the tables in this book: platform and `os-release`, interpreter and search paths, locale, tilde and `file volumes`, XDG variables, `exec` error cases, Tk with screen, scaling and mouse wheel bindings, fonts, themes and X resources, window attributes, `tk systray`, `tk sysnotify`, `tk print` and the clipboard. Each query runs on its own; if a tool is missing, the error message appears in that line.

```
tclsh8.6 messprobe-linux.tcl            # Tcl part only
env -i HOME=$HOME PATH=/usr/bin:/bin DISPLAY=$DISPLAY LANG=C.UTF-8 \
    /opt/tcl9/bin/wish9.0 messprobe-linux.tcl
wish9.0 messprobe-linux.tcl -keys       # then a window for keys
```

Differences between 8.6.14 and 9.0.4 in the container (identical values omitted):

```
                        8.6.14                      9.0.4
threaded                1                           (Schlüssel fehlt)
tcl_library             /usr/share/tcltk/tcl8.6     //zipfs:/lib/tcl/tcl_library
tk_library              /usr/share/tcltk/tk8.6      //zipfs:/lib/tk/tk_library
```

```
zipfs mount             (kein zipfs)                  //zipfs:/lib/tk
    …/libtcl9tk9.0.so
                                                      //zipfs:/lib/tcl
    …/libtcl9.0.so
file home               Fehler                        /root
file normalize ~        /root                         <Arbeitsordner>/~
file volumes            /                             //zipfs:/ /
file tempdir            Fehler                        /tmp/tcl_…
Rechte /tmp             041777                        0o41777
tk::scalingPct          fehlt                         100
tk Unterbefehle         ohne print/sysnotify/systray  mit print, sysnotify, systray
Mausrad Text            Button-4/5, MouseWheel        nur MouseWheel
TButton -padding        3 3                           2.25p
<<Copy>>                ohne XF86Copy                 mit <Key-XF86Copy>
systray                 fehlt                         angelegt, exists=1
sysnotify               fehlt                         libnotify geladen
tk::print::cups         fehlt                         Tcl-Ensemble (lpstat/lpr)
```

Identical in both: `os` Linux, `machine` x86_64, `encoding system` utf-8 under `C.UTF-8`, `windowingsystem` x11, screen 1280x1024 px, 325x260 mm, `tk scaling` 1.389, `TkDefaultFont` Inter 10, themes `clam alt default classic`, background `#d9d9d9`, class `Messprobe-linux.tcl`.

19.4 More example scripts

File	Purpose	Chapter
xdg-ordner.tcl	XDG directories and user directories, 8.6 and 9	Paths, environment, XDG
utf8-wache.tcl	UTF-8 script that works with 8.6 under LANG=C	Encoding and locale
rad-und-tasten.tcl	Trigger and read mouse wheel and modifiers with xdotool	Mouse, keyboard, clipboard
auswahl-probe.tcl	PRIMARY and CLIPBOARD, without deadlock	Mouse, keyboard, clipboard
wm-probe.tcl	Focus, frame, iconify, fullscreen, WM_CLASS	X11, window managers; tests
spaeter-rahmen.tcl	Dialog position with a late frame, once corrected vs. observed	Tests
meinapp.desktop	Template for the launcher	Native dialogs and desktop integration
gui-test-xvfb.sh	Test under xvfb-run, optionally with openbox	GUI tests with Xvfb

All `.tcl` scripts exit on their own and therefore also run under `xvfb-run`. `rad-und-tasten.tcl` needs `xdotool` and reports SKIP without it.

19.5 Howto series

Three of the series are published (chapter 1). Recorded here is how all five ran in the container and with what result:

```
cd serie-linux && tclsh8.6 run-all.tcl
cd serie-xvfb  && xvfb-run -a tclsh9.0 run-all.tcl
cd serie-xvfb  && xvfb-run -a sh -c 'openbox >/dev/null 2>&1 & sleep 1;
    ./run-all.tcl'
cd serie-xfce  && tclsh8.6 run-all.tcl          # only in an Xfce session
cd serie-tcltk-bauen && tclsh9.0 run-all.tcl
cd serie-tcl9-app    && tclsh9.0 run-all.tcl
```

Results in the container on 2026-10-20 (runs with `env -i`):

Series	8.6.14	9.0.4
serie-linux	10 OK, 2 FAIL (03: no USER; 05: no xdg-user-dir)	9 OK, 3 FAIL (additionally 02: tilde)
serie-xvfb (without window manager)	19 OK, 1 SKIP (10: iconify)	19 OK, 1 SKIP
serie-tcltk-bauen	11 OK, 1 SKIP (10: Itcl missing)	11 OK, 1 FAIL (03: no -ltommath)
serie-tcl9-app	not applicable	18 OK
serie-xfce	SKIP (no Xfce)	not run

The FAILs are differences in the environment, not errors in the series: `env -i` does not set USER, Ubuntu does not have `xdg-user-dirs` installed, Tcl 9 does not expand the tilde (see Paths, environment, XDG), and the source build links libtommath in statically (see Building Tk: dependencies).

Next chapter: Glossary

20 Glossary

20.1 Symbols

.desktop file
: Launcher according to the Desktop Entry Specification, in `~/.local/share/applications/` or `/usr/share/applications/`. Keys `Exec`, `Icon`, `StartupWMClass`.

//zipfs:/
: Root of the Tcl 9 ZIP file system. In a source build the Tcl library lives under `//zipfs:/lib/tcl`, appended to `libtcl9.0.so`. Under Tcl 9 also listed in `file volumes`.

20.2 A

auto_execok
Looks up a program in PATH, returns a list or an empty result. Caches every answer in `auto_execs`.

auto_path
Search list for `package require`. Debian: `/usr/share/tcltk` and `/usr/lib/tcltk/<multiarch>`. Source build under `/opt/tcl9`: only zipfs and `/opt/tcl9/lib`.

20.3 C

CLIPBOARD
The X11 selection for copy and paste. Set by `<<Copy>>` or `clipboard append`. Disappears with its owner unless a manager takes it over.

Command
Modifier name in `bind`. On Aqua the Command key. On X11 equal to Mod1 (Alt) in Tk 8.6, equal to Meta in Tk 9.0.4.

container check
In this book: check in an Ubuntu 24.04 container on 2026-10-20 with Tcl/Tk 8.6.14 (package) and 9.0.4 (`/opt/tcl9`), Xvfb without a window manager.

20.4 D

DBus
Message bus of the session. `xfconf-query`, `notify-send` and libnotify need it. Address in `DBUS_SESSION_BUS_ADDRESS`.

DISPLAY
Environment variable with the address of the X server, such as `:0` or `:99`. Without it Tk does not start under X11.

20.5 E

encoding system
Default encoding of Tcl. On Linux taken from `LC_ALL`, `LC_CTYPE`, `LANG`. `C` and no locale: iso8859-1.

EWMH
Extended Window Manager Hints, the `_NET_WM_*` properties. Tk uses them to set fullscreen, `-zoomed`, `-type`, opacity and the window icon.

exo-open
Xfce tool that starts preferred applications by role (`--launch FileManager`). Under Xfce, `xdg-open` passes on to it.

20.6 F

fontconfig
Library that maps font names to files. Tk asks for `sans-serif` and `monospace`. `fc-match` shows the result.

forky
Code name of Debian 14, currently testing. Test system of the series.

20.7 L

libnotify
: Library for desktop notifications. Tk 9 loads `libnotify.so.4` at runtime for `tk sysnotify`.

locale
: Language and character set setting from `LC_ALL`, `LC_CTYPE`, `LANG`. On Linux it determines `encoding system`.

20.8 M

Mod1, Mod4
: X11 modifier bits. In the default layout Mod1 = Alt, Mod4 = Super.

MouseWheel
: Wheel event in Tk. Under X11 only from Tk 9 on; Tk 8.6 reports the wheel as `<Button-4>` and `<Button-5>`.

multiarch
: Debian layout for libraries per architecture, such as `/usr/lib/x86_64-linux-gnu`. The real `tclConfig.sh` lives there.

20.9 O

os-release
: File `/etc/os-release` with `ID`, `ID_LIKE`, `VERSION_CODENAME`. Source for the distribution.

20.10 P

PRIMARY
: The X11 selection of the most recently selected text. Pasted with the middle mouse button (`<<PasteSelection>>`).

20.11 R

reparenting
: A window manager puts the window into a frame of its own. Recognizable by `wm frame.` and `winfo id.` being different.

RUNPATH
: Search path for libraries, recorded in an ELF file. The source build sets `/opt/tcl9/lib`.

20.12 S

scalingPct
: `::tk::scalingPct`, Tk 9: resolution as a percentage of 96 dpi, rounded to 25, at least 100.

StartupWMClass
: Key in a `.desktop` file. Must match the class from `WM_CLASS`.

20.13 T

TCL9_0_TM_PATH
: Additional tm paths for Tcl 9.0, also read by 9.1. Separated by colons, inserted in reverse order. For 8.6: `TCL8_6_TM_PATH`.

TCLLIBPATH
Environment variable whose entries are put at the front of `auto_path`. A Tcl list, separated by spaces.

tk scaling
Pixels per point. At 100 dpi 1.389.

20.14 W

Wayland
Display protocol, successor to X11. Tk 9.0.4 has no Wayland backend and runs there via XWayland.

window manager
Program that provides frame, title bar, focus and iconify under X11. Under Xfce, xfwm4. Xvfb has none.

WM_CLASS
X property with the name and class of a window. Tk: program name and program name with a capital first letter.

20.15 X

X resources
Settings database of the X server (`xrdb`). Tk reads it at startup. `*TkTheme` selects the ttk theme.

X11
Windowing system of Tk on Linux. `tk windowingsystem` returns `x11`.

XDG
Freedesktop specifications. Here mainly Base Directory (`XDG_CONFIG_HOME` and related) and `user-dirs`.

xdg-open
Opens a file or address with the default application. Package `xdg-utils`.

xdg-user-dir
Returns the localized path of a user directory (`DOCUMENTS`). Package `xdg-user-dirs`.

Xfce
Desktop on the author's workstation, version 4.20.

xfconf
Settings service of Xfce, read and written with `xfconf-query`.

Xft
X FreeType, Tk's font rendering under X11. `tk::pkgconfig get fontsystem` returns `xft`.

Xft.dpi
X resource for the font DPI. Tk 9 derives `tk scaling` from it, Tk 8.6 does not.

Xvfb
X server without a screen and without a window manager. For tests. `xvfb-run` starts it for one command.

XWayland
X server that a Wayland compositor provides for X11 programs.

20.16 Z

zipkit
: Statically built interpreter with the Tcl library appended, the basis for distributable `zipfs mkimg` programs.

Next chapter: Error messages

21 Error messages

Messages that occurred during the checks for this book, with cause and remedy. Unless stated otherwise, identical in the container under 8.6.14 and 9.0.4.

21.1 no display name and no $DISPLAY environment variable

```
application-specific initialization failed: no display name and no $DISPLAY
    environment variable
```

DISPLAY is missing. `wish` then runs the script without Tk and exits with 0. Remedy: set a display or use `xvfb-run`. In the script, check `package require Tk` with `catch`. See X11, window managers and Wayland.

21.2 couldn't connect to display

```
application-specific initialization failed: couldn't connect to display ":77"
```

DISPLAY is set, but no server answers. Typical after a terminated Xvfb whose socket was left behind under `/tmp/.X11-unix/`, or with `ssh` without `-X`.

21.3 couldn't execute … no such file or directory

```
couldn't execute "gibtsnicht": no such file or directory
```

exec does not find the program. `errorCode` is `POSIX ENOENT {no such file or directory}`. Use `auto_execok` first.

21.4 child process exited abnormally

The child exited with a value other than 0. `errorCode` is `CHILDSTATUS <pid> <wert>`. Not an error in the program's sense if the value is an answer (for example `grep` without a match: 1).

21.5 child killed: software termination signal

The child was terminated with `SIGTERM`. `errorCode`: `CHILDKILLED <pid> SIGTERM {software termination signal}`.

21.6 (text on stderr)

exec reports as error text whatever the child wrote to stderr, even with exit status 0. `errorCode` is `NONE`. Remedy: `-ignorestderr` or `2>@1`. See exec, xdg-open, processes.

21.7 invalid or incomplete multibyte or wide character

```
error reading "file5": invalid or incomplete multibyte or wide character
```

Tcl 9 only: a channel with `-encoding utf-8` has read bytes that are not UTF-8. Tcl 8.6 silently interprets them as Latin-1. See Encoding and locale.

21.8 unknown or ambiguous subcommand "home"

```
unknown or ambiguous subcommand "home": must be atime, attributes, …
```

`file home`, `file tildeexpand` and `file tempdir` exist only from Tcl 9 on. Under 8.6 use `$env(HOME)`.

21.9 unknown or ambiguous subcommand "systray"

```
unknown or ambiguous subcommand "systray": must be appname, busy, caret,
    fontchooser, inactive, scaling, useinputmethods, or windowingsystem
```

`tk systray`, `tk sysnotify` and `tk print` exist only from Tk 9 on.

21.10 bad button number "8"

Tk 8.6 only knows buttons 1 to 5 in `bind`. Tk 9 also accepts 6 to 9.

21.11 bad event type or keysym "Super"

There is no modifier `Super`. In `bind` the Super key is called `Mod4`.

21.12 CLIPBOARD selection doesn't exist or form "STRING" not defined

```
CLIPBOARD selection doesn't exist or form "STRING" not defined
PRIMARY selection doesn't exist or form "STRING" not defined
```

Nobody owns the selection. For CLIPBOARD: nothing was copied, or the program that copied has exited. For PRIMARY: nothing is selected. Always wrap `selection get` and `clipboard get` in `catch`.

21.13 Error: target STRING not available

Message from `xclip` when the owner of the selection is no longer running. In the container after `wish` has exited.

21.14 Unable to obtain the list of printers

```
Unable to obtain the list of printers. Command "lpstat" not found. Please install
    the CUPS package for your system.
```

Tk 9, `::tk::print::cups getprinters` without CUPS. Install the package `cups-client` or `cups`. Only 9.0.4.

21.15 Failed to execute child process "dbus-launch"

```
Failed to execute child process "dbus-launch" (No such file or directory)
```

`notify-send` without a session DBus. Exit status 1. In a desktop session this does not happen; in cron, `ssh` or a container it does. `tk sysnotify` reports nothing in the same situation.

21.16 Verbindung ist gescheitert

`xfconf-query` without a session DBus (`serie-xfce`, reference; German message on the author's workstation). Applies to every call.

21.17 SKIP: serie-xfce braucht XFCE

```
SKIP: serie-xfce braucht XFCE (XDG_CURRENT_DESKTOP=)
```

No Xfce session. The series then checks nothing.

21.18 WARNING: unrecognized options

The Tcl 9 `configure` no longer knows the switch; threads are always on (`serie-tcltk-bauen`, howto 05). Omit the switch.

21.19 file format not recognized

The linker hits an empty `.o` from an aborted `make` (`serie-tcltk-bauen`, howto 11). `find . -name '*.o' -size 0 -delete`, then run `make` again.

21.20 XInput2.h missing

Missing header `X11/extensions/XInput2.h` in the Tk source build, reported in *Tcl/Tk 9.0 bauen* on Ubuntu 24.04. Package `libxi-dev`. The building book does not give the exact wording of the message; it has not been verified here.

21.21 this extension is compiled for Tcl 8.6

```
% package require sqlite3
this extension is compiled for Tcl 8.6
    while executing
"load /usr/lib/tcltk/sqlite3/libsqlite3.53.4.so"
```

Under Debian's `tclsh9.0` and `tclsh9.1` with `libsqlite3-tcl` up to 3.53.4-2; these versions are built for Tcl 8.6 only. Since 3.54.1-1 the package ships a library for Tcl 9, and the message no longer appears. Background and a way for older versions in Search paths, section *sqlite3 under Debian's Tcl 9*.

21.22 Keine Berechtigung (permission denied) when installing to /usr/bin

```
/bin/bash ../tclconfig/install-sh -c  -m 755 ../compat/sqlite3/spaceanal.tcl
    "/usr/bin/sqlite3_analyzer"
cp: reguläre Datei '/usr/bin/_inst.669779_' kann nicht angelegt werden: Keine
    Berechtigung
```

A TEA package built against Debian's `tclConfig.sh` adopts the prefix `/usr`. `make install-binaries` then wants to put programs into `/usr/bin`. Library and index are already installed at this point. Remedy: `make install-lib-binaries`.

Next chapter: Sources

22 Sources

22.1 Introduction

The book explains in the text where a statement comes from. This appendix lists the sources in full. Third-party texts are summarized, not translated or copied. Short verbatim passages, such as error messages and commands, are shown in code blocks.

22.2 Own runs

22.2.1 The author's howto series

The series are the author's companion material; `serie-linux`, `serie-xvfb` and `serie-xfce` are under Serien/. Each contains `README.md` with the section "Gemessen" (measured), `TUTORIAL.md`, `howto/00-referenz.md` and, for each howto, an explanation `NN-*.md` with a demo `NN-*.tcl`; `run-all.tcl` starts all demos.

Series	Howtos	As of, per README	Used in
serie-linux	12	Linux, Tcl 8.6.18, Debian forky; no date	tcl_platform, paths, encoding, exec, dialogs
serie-xvfb	20	Tcl/Tk 8.6.14 and 9.0.4 under Xvfb, without a window manager and with openbox; no date	X11, both test chapters
serie-xfce	12	Xfce 4.20, Debian, X11; no date	Xfce, desktop integration
serie-tcltk-bauen	12	Debian forky, 2026-09-15, tclsh8.6 and tclsh9.0, 12/12 each	Which Tcl, building Tk, search paths
serie-tcl9-app	18	Tcl 9.0.x	Distributing

The values for Xfce with a 29-pixel title bar come from `serie-xvfb` (howtos 04 and 13), checked on the author's workstation.

22.2.2 Container check

On 2026-10-20 in a container with Ubuntu 24.04.5 LTS, kernel 6.18.44, x86_64, glibc 2.39.

- Tcl/Tk 8.6.14 from the Ubuntu packages `tcl8.6`, `tk8.6`.
- Tcl/Tk 9.0.4 as a source build under `/opt/tcl9`, sources next to it under `tcl9.0.4` and `tk9.0.4`.
- Xvfb 21.1.12 without a window manager, 1280x1024x24, 100 dpi; for scaling additionally 1920x1080 with `-dpi 192`; for X resources with `-noreset`.
- Tools: `xdotool`, `xclip`, `xprop`, `xrdb`, `xmodmap`, `readelf`, `objdump`, `notify-send` 0.8.3 (libnotify 0.8.3).
- Runs with `env -i` for search paths and encoding.
- Scripts: `beispiele/messprobe-linux.tcl`, `rad-und-tasten.tcl`, `auswahl-probe.tcl`, `wm-probe.tcl`, `spaeter-rahmen.tcl`, `utf8-wache.tcl`, `xdg-ordner.tcl`; plus all series except `serie-xfce` (see CLI).

22.2.3 Check on lxpro

On the author's workstation, Debian unstable (forky), October 2026, run by the author:

- Debian packages `tcl8.6` 8.6.18, `tcl9.0` 9.0.4, `tcl9.1` 9.1.0, `tk9.0` 9.0.4, `tk9.1` 9.1.0,

libsqlite3-tcl 3.53.4-2, from 2026-11-03 on 3.54.1-1 from unstable.
- BAWT builds 9.0.1 and 9.0.4 under /opt/Tcl, source build 9.1.0 under /opt/tcl91.
- sqlite3 3.53.4 from sqlite3.53.4.tar.gz of the Tcl 9.1.0 distribution, built against Debian's tcl8.6-dev, tcl9.0-dev and tcl9.1-dev.
- Reports with examples/demo-tuappinfo.tcl from tclutils 0.69.0.
- Check of the open points on 2026-11-02 under Xfce 4.20, X11, wish 8.6.18, 9.0.4 and 9.1.0: os-release, wm frame, -alpha, scalingPct on the dialog, <Command-c>, Super, xfce4-clipman, zenity, desktop-file-validate, tk systray, Xfce detection, grab -global, readelf -d.

22.2.4 What was missing in the container

Installed additionally for XWayland: weston 13.0.0, xwayland, x11-apps (Ubuntu packages), Weston nested inside Xvfb.

Not present in the container and therefore not checked: desktop session, session DBus, Xfce, window manager, Wayland, CUPS, xdg-user-dirs, desktop-file-utils, zenity, kdialog, Debian itself.

22.3 Tcl and Tk

- **Tcl/Tk source code 9.0.4**, read in the container:
  - tk/generic/tkEvent.c: translation of buttons 4 to 7 into MouseWheel.
  - tk/generic/tkBind.c: modifier table (Command, Option), button numbers from 4 on with event generate.
  - tk/unix/tkUnixKey.c: determining the Meta and Alt bits.
  - tk/unix/tkUnixWm.c: WM_CLASS, _NET_WM_*.
  - tk/unix/tkUnixEvent.c: fallback to 75 dpi under XWayland.
  - tk/unix/tkUnixSysNotify.c, tk/unix/tkUnixSysTray.c, tk/library/systray.tcl: tk sysnotify, tk systray.
  - tk/library/print.tcl: tk print under X11.
  - tk/library/scaling.tcl: scaling under X11.
  - tk/library/ttk/fonts.tcl: standard fonts.
  - tk/library/tkfbox.tcl: file dialog, hidden files.
  - tcl/library/init.tcl, tcl/library/auto.tcl: auto_execok, auto_reset.
  - tcl/library/msgcat/msgcat.tcl: locale variables.
  - tcl/library/tcltest/tcltest.tcl (2.5.11): return value of runAllTests.
  - tcl/unix/Makefile.in, tk/unix/Makefile.in: installed programs.
    https://core.tcl-lang.org/tcl/, https://core.tcl-lang.org/tk/
- **Tcl/Tk manual pages** 9.0: bind, event, selection, clipboard, wm, winfo, tk, file, exec, encoding, zipfs, tm. https://www.tcl-lang.org/man/tcl9.0/
- **TIP 602**: no longer expand the tilde in the file system.
  https://core.tcl-lang.org/tips/doc/trunk/tip/602.md
- **TIP 656**: encoding profiles, strict as the default.
  https://core.tcl-lang.org/tips/doc/trunk/tip/656.md
- **Tcl/Tk 9.1.0**, download page with release notes and the bundled packages, among them sqlite3.53.4.tar.gz. https://sourceforge.net/projects/tcl/files/Tcl/9.1.0/

22.4 Debian

- **tcltk-defaults**: default version of Tcl/Tk in Debian. https://tracker.debian.org/pkg/tcltk-defaults

---

22.5 Specifications

- **XDG Base Directory Specification**, freedesktop.org.
  https://specifications.freedesktop.org/basedir-spec/latest/
- **Desktop Entry Specification**, freedesktop.org.
  https://specifications.freedesktop.org/desktop-entry-spec/latest/
- **Extended Window Manager Hints (EWMH)**, freedesktop.org.
  https://specifications.freedesktop.org/wm-spec/latest/
- **xdg-user-dirs**, freedesktop.org. https://www.freedesktop.org/wiki/Software/xdg-user-dirs/
- **os-release**, systemd manual page `os-release(5)`.
- Manual pages: `Xvfb(1)`, `xvfb-run(1)`, `xrdb(1)`, `xprop(1)`, `xmodmap(1)`, `xclip(1)`, `xdotool(1)`, `xdg-open(1)`, `notify-send(1)`, `fc-match(1)`, `readelf(1)`, `ld.so(8)`, `locale(7)`.

22.6 Companion books in the same series

Book	Used for
Debian unter Linux (book-linux-debian), chapters Tcl unter Linux, Debian erkennen, Suite und Codename	Location of the Tcl installations on lxpro and decoder, rules for /opt, os-release
Tcl/Tk unter macOS (book-macos-tcltk)	Structure and style; comparison with Aqua
Tcl/Tk 9.0 bauen (book-tcltk9.0-bauen), Tcl/Tk 9.1 bauen (book-tcltk9.1-bauen)	Unix dependencies, libxi-dev, multiple installations, static builds
Tcl-Anwendungen ausliefern (book-tcl-ausliefern), in the text Ausliefern	zipfs, zipkits, .deb and .rpm
C-Erweiterungen (book-tcl-erweiterungen)	TEA against tclConfig.sh
Pakete und Module (book-package-module)	auto_path, .tm
Tcl/Tk-Versionen nebeneinander (book-tcltk-versionen)	same package version, binary extensions per version
tclutils (book-tclutils)	tuappinfo
Tcl/Tk 9.1 (book-tcltk9.1)	tk fileicon, BiDi under X11
Encoding (book-encoding)	Profiles, TIP 656
Testen (book-tcl-testen)	tcltest
Printing (book-printing)	Printing via CUPS

22.7 Contradictions between the sources

Where results differ, the chapter says so. The most important ones:

- *Debian unter Linux*, chapter *Tcl unter Linux*: "<Command-c> … so on Linux responds to Alt+C." True for Tk 8.6; under Tk 9.0 and 9.1 no key at all, in the container and on the Xfce workstation. See Mouse, keyboard, clipboard.
- `serie-linux`, README: `file volumes` returns `/`. Under Tcl 9 additionally `//zipfs:/`.
- `serie-tcltk-bauen`, howto 03: `TCL_LIBS` of 9.0.4 contains `-ltommath`. True for the Debian package, not for the source build.
- `serie-tcltk-bauen`, howtos 04 and 07: Tk 9 additionally links `libpng`. In the container and on the workstation, `libpng16` appears for all Tk versions only via freetype (`readelf -d`).
- *Tcl/Tk 9.0 bauen*: `libxi-dev` needed because of `XInput2.h`. In the Tk 9.0.4 source code, `XInput2` does not occur.

22.8 Trademarks

Linux is a trademark of Linus Torvalds. Debian is a trademark of Software in the Public Interest, Inc. Ubuntu is a trademark of Canonical Ltd. Xfce, X.Org and freedesktop.org are names of their respective projects. Tcl and Tk are under a BSD-style license. This book is not affiliated with these organizations.

End of the book.

23 Test environment

23.1 Introduction

This appendix lists the machines and versions on which the examples in this book were tried, the check scripts for them and the points that are still open. The chapters mention only what is needed for understanding. How the values were obtained is described in Appendix E: Sources.

23.2 Machines and versions

Two environments, both x86_64 with X11:

What	The author's Debian workstation	Container
System	Debian forky (testing)	Ubuntu 24.04.5 LTS, kernel 6.18, glibc 2.39
Tcl 8.6	8.6.18 from the package	8.6.14 from the packages tcl8.6, tk8.6
Tcl/Tk 9	9.0.4 and 9.1.0 from the packages	9.0.4, source build under /opt/tcl9
Display	Xfce 4.20 under X11 (window manager xfwm4)	Xvfb without a window manager, 1280x1024x24, 100 dpi
Source of the values	the five howto series	beispiele/and the series except serie-xfce

In the container, the interpreters for search paths and encoding run with an empty environment (`env -i`), so that no variable from the session distorts the values. Statements about Tk internals (mouse wheel, `tk print`, `tk systray`, scaling) were looked up in the Tk 9.0.4 sources and verified in the container. The values with a window manager come from `serie-xvfb` (openbox) and from the author's workstation (Xfce).

The container has no desktop session, no session DBus, no Xfce, no window manager, no Wayland and no CUPS. The Xfce values therefore come only from `serie-xfce`.

23.3 Check scripts

Script	checks	Chapter
beispiele/messprobe-linux.tcl	all values in the tables of this book	all, see CLI
beispiele/rad-und-tasten.tcl	mouse wheel and modifiers with xdotool	Mouse, keyboard, clipboard
beispiele/auswahl-probe.tcl	PRIMARY and CLIPBOARD, without deadlock	Mouse, keyboard, clipboard
beispiele/wm-probe.tcl	focus, frame, iconify, fullscreen, WM_CLASS	X11, window managers; tests
beispiele/spaeter-rahmen.tcl	dialog position with a late frame	Tests
beispiele/utf8-wache.tcl	UTF-8 script under LANG=C with 8.6	Encoding and locale
beispiele/xdg-ordner.tcl	XDG directories and user directories, 8.6 and 9	Paths, environment, XDG
serie-*/run-all.tcl	all howtos of a series (three published, chapter 1)	see chapter 1

23.4 Not yet tested

Chapter	open
X11, window managers and Wayland	a real Wayland session (nested Weston checked)
Native dialogs and desktop integration	tk systray in the Xfce panel, visually
What the test sets up itself	the howtos on Windows

A result for one of these points belongs in the respective chapter and is removed from here.

24 License

Copyright (c) 2026 Gregor Ebbing

Text: Creative Commons Attribution-ShareAlike 4.0 International
(CC BY-SA 4.0)
Code: MIT License (below)
Third-party material: license of the original

Scope

The following licenses apply only to content for which the author
holds the rights. Material taken from other projects is excluded
(see "Third-party material").

Text

Unless stated otherwise, all text written by the author for this book
is licensed under the Creative Commons license
"Attribution-ShareAlike 4.0 International" (CC BY-SA 4.0). This
applies in particular to the chapters, appendices, README and other
documentation.

You are free to copy and redistribute the material in any medium or
format, and to remix, transform and build upon it for any purpose,
even commercially, under the following terms:

* Attribution: You must give appropriate credit, provide a link to
  the license, and indicate if changes were made.

* ShareAlike: If you remix, transform, or build upon the material,
  you must distribute your contributions under the same license as
  the original.

The full legal code is binding:
https://creativecommons.org/licenses/by-sa/4.0/legalcode

Code

The programs and scripts written by the author for this book
(examples, test scripts, tools) are licensed under the MIT License.

Code examples within the text are additionally licensed under the
MIT License. They may be used in your own code without the conditions
of the CC license.

Third-party material

```
Excerpts from other projects, such as examples from the Tcl/Tk,
tcllib or Tablelist manual pages, are marked with their source and
remain under the license of the original as stated by the respective
project (for Tcl/Tk and tcllib the file license.terms). Larger
excerpts, if any, are listed with source, copyright holder and
license in THIRD-PARTY.md.

---

MIT License

Copyright (c) 2026 Gregor Ebbing

Permission is hereby granted, free of charge, to any person obtaining
a copy of this software and associated documentation files (the
"Software"), to deal in the Software without restriction, including
without limitation the rights to use, copy, modify, merge, publish,
distribute, sublicense, and/or sell copies of the Software, and to
permit persons to whom the Software is furnished to do so, subject to
the following conditions:

The above copyright notice and this permission notice shall be
included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS
OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.
IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY
CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT,
TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE
SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
```

Index

-

-noreset 48

.

.desktop 42

/

/etc/os-release 8
/proc 10
/usr/share/tcltk 18

:

::tk::scalingPct 35

A

auto_execok 26
auto_path 18

C

Command 39
CUPS 44

D

DISPLAY 30

E

encoding system 23
EWMH 31

G

grab 51

I

info loaded 23

L

libsqlite3-tcl 20
libxi-dev 54

M

MouseWheel 37

P

PRIMARY 39

R

reparenting 31
resource database 36

S

StartupWMClass 43

T

tcl9.0 11
TCL9_0_TM_PATH 22
tcl_platform 8
TCLLIBPATH 21
threaded 9
tk scaling 34
tk sysnotify 43

W

Wayland 32
window manager 29
winfo viewable 50
WM_CLASS 30

X

X11 7
XDG 7
XDG Base Directory 15
xdg-open 28
Xfce 44
xfconf 45
Xft 34
Xft.dpi 35
xfwm4 47
Xvfb 47
xvfb-run 47

Z

zipfs 12, 56
zipkit 58

~

~/.config/user-dirs.dirs 17